

2026 Summer Break FTL Study

Basic Lecture

2026.07.21

Presentation by Choi, Gunhee

choi_gunhee@dankook.ac.kr

Contents

1. Address Translation
 2. Wear-Leveling
 3. Garbage Collection
 4. TRIM
 5. FEMU
- Appendix

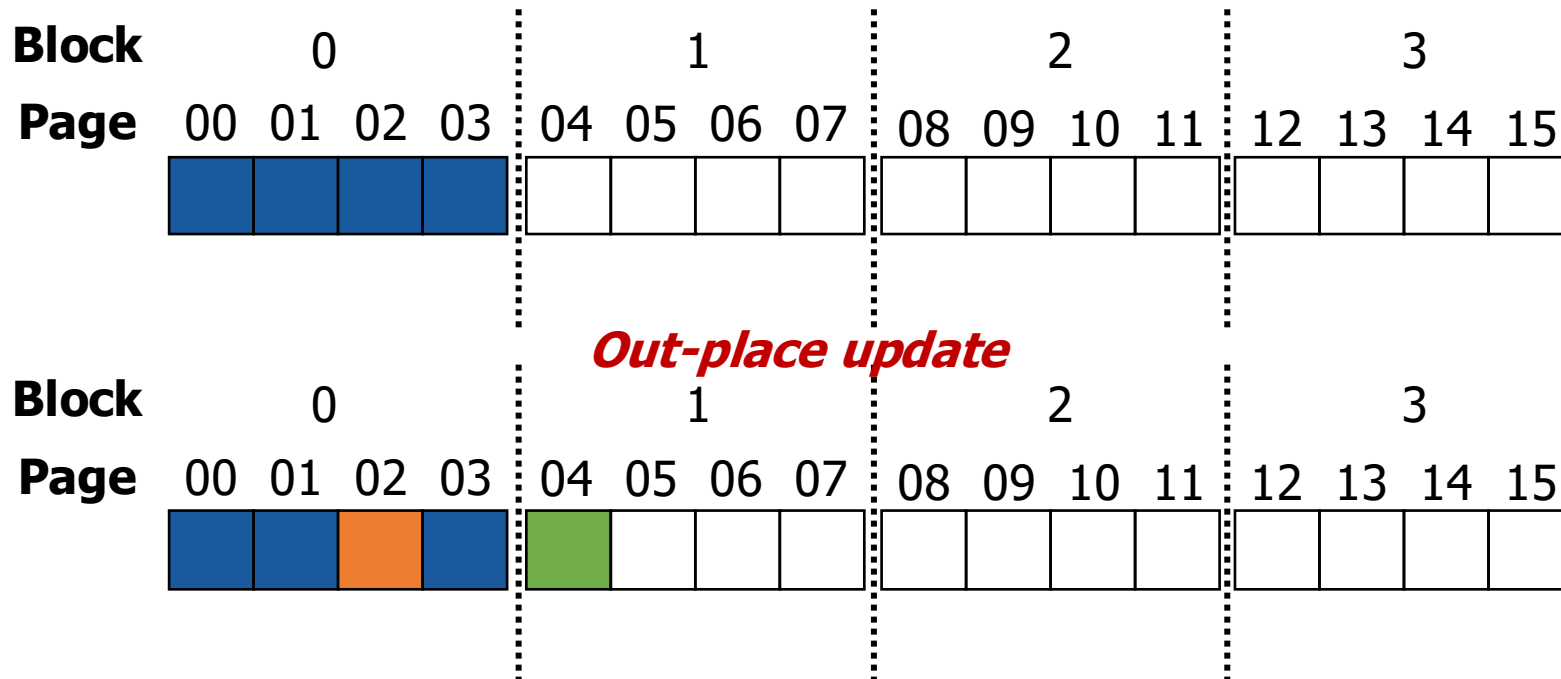
1. Address Translation

- 주소 변환(Address Translation)의 필요성

- 1) Read/Write/Erase 단위 불일치
- 2) Overwrite(덮어쓰기) 불가
- 3) 기존 단위 유지 필요성 (Legacy)

→ Out-place update

✓ Page 02 Request for modification



1. Address Translation

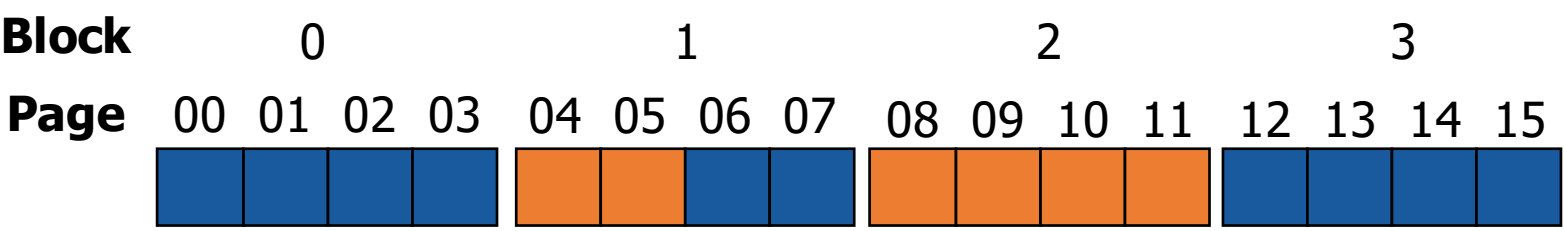
- Page mapping algorithm

✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

Page Mapping Table

LPN	PPN
00	0
01	1
02	2
03	3
04	4→5→7
05	6
06	-
07	-
08	8→12
09	9→13
10	10→14
11	11→15
12	-
13	-
14	-
15	-



1. Address Translation

- Page mapping algorithm

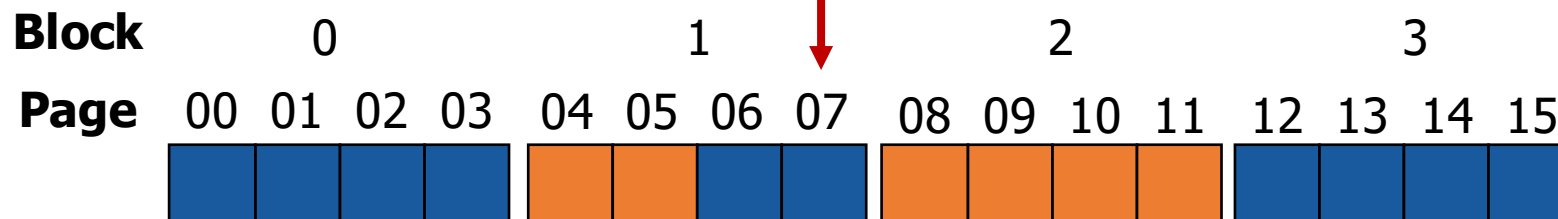
- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

Logical Page Address : Page 4

Page Mapping Table

LPN	PPN
00	0
01	1
02	2
03	3
04	4→5→7
05	6
06	-
07	-
08	8→12
09	9→13
10	10→14
11	11→15
12	-
13	-
14	-
15	-



■ Valid page ■ Invalid page □ Erased page (Free page)

1. Address Translation

- Page mapping algorithm

- ✓ Mapping table size calculation

- ✓ SSD Capacity: 1TB

- ✓ Page size : 4KB

- ✓ Entry size : 4Bytes

- SSD capacity : 1TB $\rightarrow 2^{40}$ bytes

- Page size : 4KB $\rightarrow 2^2 * 2^{10} \rightarrow 2^{12}$ bytes

- # of pages = SSD capacity / Page size
 $= 2^{40} / 2^{12} = 2^{28}$ pages

- Entry = 2^{32}

- Table size = # of page * Entry size
 $= 2^{28} * 4\text{Bytes}$
 $= 2^{30} = \mathbf{1GB}$

Page mapping table의 크기가 너무 크다.

1. Address Translation

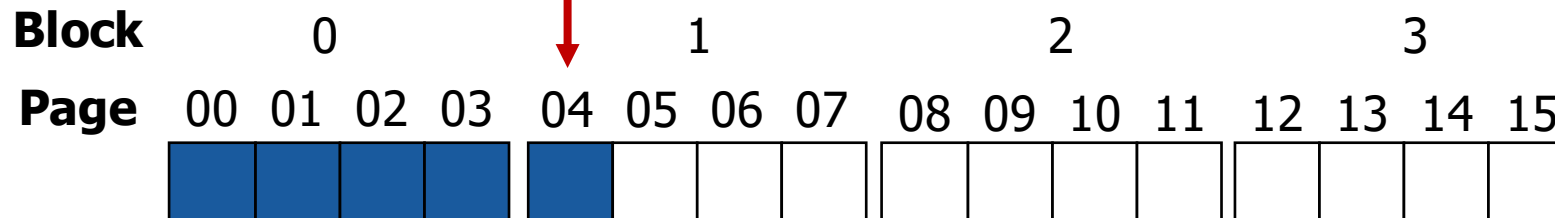
- Block mapping algorithm

- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = LPN / \# \text{ of pages per block}$
 $= 4 / 4 = 1$

✓ $\text{Page offset} = LPN \% \# \text{ of pages per block}$
 $= 4 \% 4 = 0$



Block Mapping Table

LBN	PBN
00	0
01	1
02	-
03	-

1. Address Translation

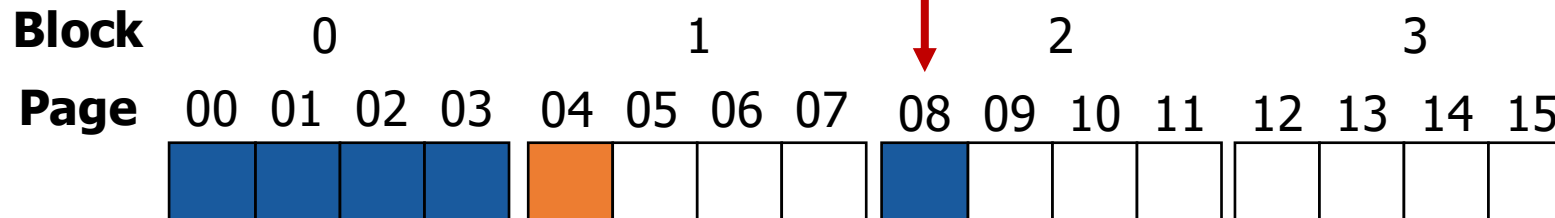
- Block mapping algorithm

- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = LPN / \# \text{ of pages per block}$
 $= 4 / 4 = 1$

✓ $\text{Page offset} = LPN \% \# \text{ of pages per block}$
 $= 4 \% 4 = 0$



Block Mapping Table

LBN	PBN
00	0
01	1 → 2
02	-
03	-

1. Address Translation

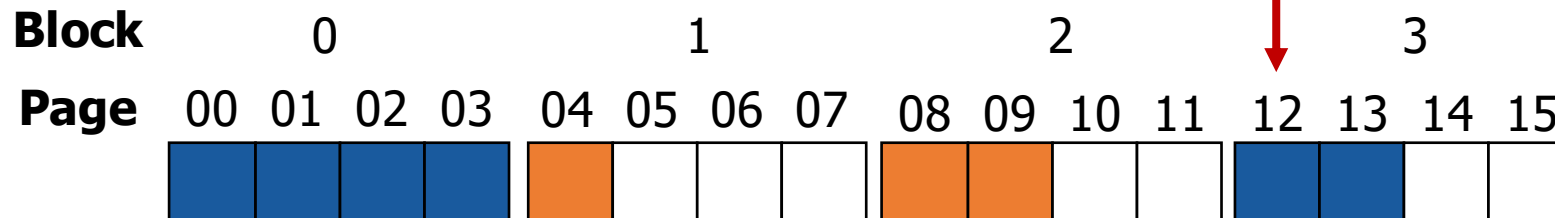
- Block mapping algorithm

- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = LPN / \# \text{ of pages per block}$
 $= 4 / 4 = 1$

✓ $\text{Page offset} = LPN \% \# \text{ of pages per block}$
 $= 4 \% 4 = 0$



Block Mapping Table

LBN	PBN
00	0
01	1 → 2 → 3
02	-
03	-

1. Address Translation

- **Block mapping algorithm**

- ✓ Mapping table size calculation

- ✓ SSD Capacity: 1TB

- ✓ Page size : 4KB

- ✓ Entry size : 4Bytes

- ✓ Block size : 2MB

- SSD capacity : 1TB $\rightarrow 2^{40}$ bytes

- Block size : 2MB $\rightarrow 2 * 2^{20} \rightarrow 2^{21}$ bytes

- # of Blocks = SSD capacity / Block size
 $= 2^{40} / 2^{21} = 2^{19}$ pages

- Entry = 2^{32}

- Table size = # of block * Entry size
 $= 2^{19} * 4\text{Bytes}$
 $= 2^{21} = \mathbf{2MB}$

너무 잦은 block copy가 발생함.

1. Address Translation

- Hybrid Mapping Schemes

- 기존 방식 장점

- Page mapping → 성능 우수
 - Block mapping → Table size가 작음

- 기존 방식 단점

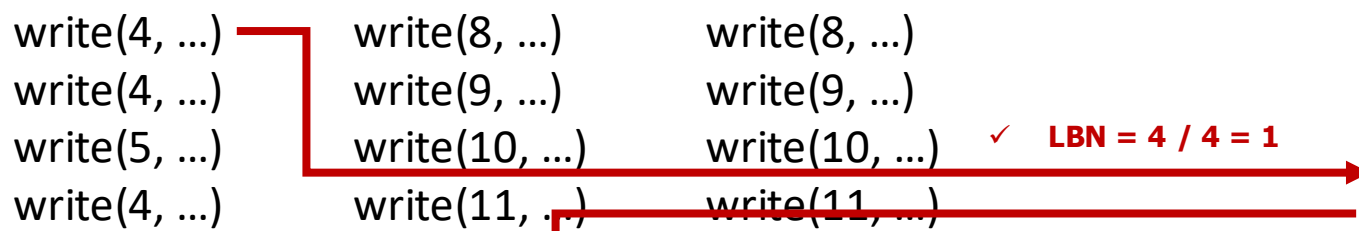
- Page mapping → Table size가 너무 큼.
 - Block mapping → Block copy 자주 발생함.

Page mapping + Block mapping

1. Address Translation

- BAST algorithm (Block-Associative Sector Translation)

✓ Write Requests

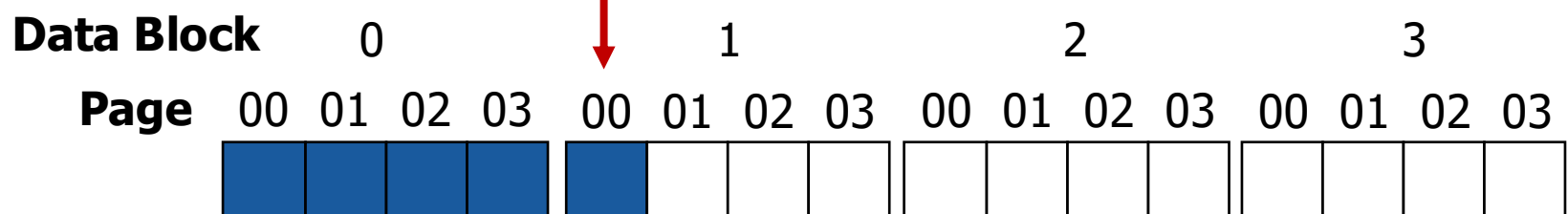
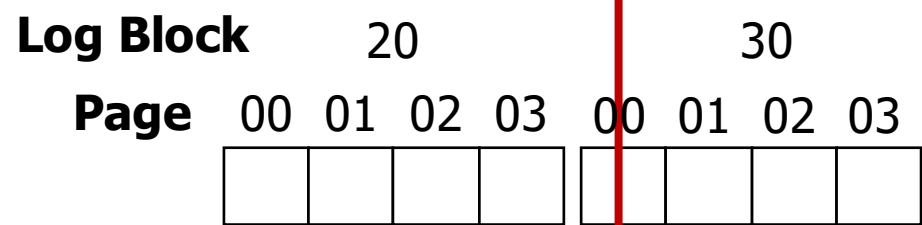


Block Mapping Table

LBN	PBN
00	
01	1
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
00		
01		
02		
03		



1. Address Translation

- BAST algorithm (Block-Associative Sector Translation)

✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = 4 / 4 = 1$

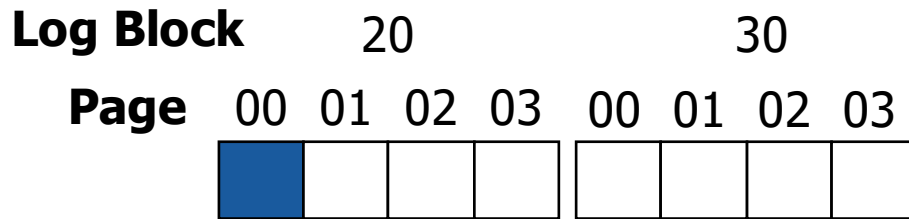
✓ $offset = 4 \% 4 = 0$

Block Mapping Table

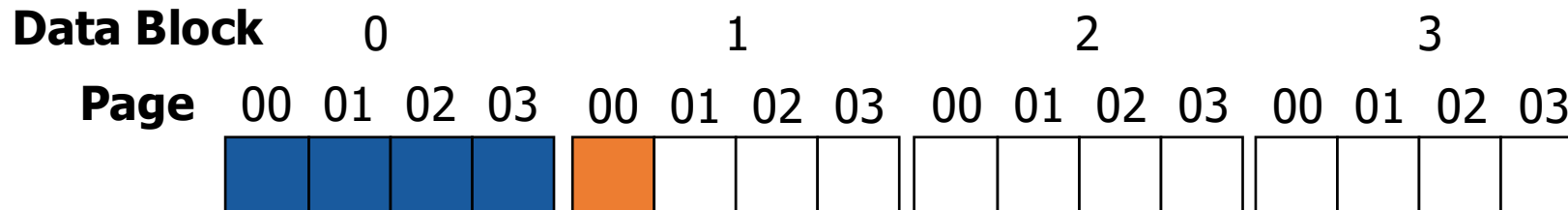
LBN	PBN
00	
01	1
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4}



**Data block : Log block
→ 1:1**



Valid page
 Invalid page
 Erased page (Free page)

1. Address Translation

- BAST algorithm (Block-Associative Sector Translation)

✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = 5 / 4 = 1$

✓ $offset = 5 \% 4 = 1$

Block Mapping Table

LBN	PBN
00	
01	1
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4}

Log Block

20

30

Page

00	01	02	03	00	01	02	03

Data Block

0

1

2

3

Page

00	01	02	03	00	01	02	03	00	01	02	03	00	01	02	03

Valid page Invalid page Erased page (Free page)

1. Address Translation

- BAST algorithm (Block-Associative Sector Translation)

- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

✓ $LBN = 4 / 4 = 1$

✓ $offset = 4 \% 4 = 1$

Block Mapping Table

LBN	PBN
00	
01	1
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,4}

Log Block

20

30

Page

00	01	02	03	00	01	02	03

Data Block

0

1

2

3

Page

00	01	02	03	00	01	02	03	00	01	02	03	00	01	02	03

Valid page
 Invalid page
 Erased page (Free page)

1. Address Translation

- BAST algorithm (Block-Associative Sector Translation)

- ✓ Write Requests

write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	2
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,4}
02	30	{8,9,10,11}

✓ $LBN = 8 / 4 = 2$

Log Block

20

30

Page 00 01 02 03 00 01 02 03



Data Block

0

1

2

3

Page 00 01 02 03 00 01 02 03 00 01 02 03 00 01 02 03



Valid page
 Invalid page
 Erased page (Free page)

1. Address Translation

- Merge

✓ Data block과 log block에 분산된 valid/invalid page를 정리하여, 한 개의 data block으로 만드는 과정

- 1) Switch Merge

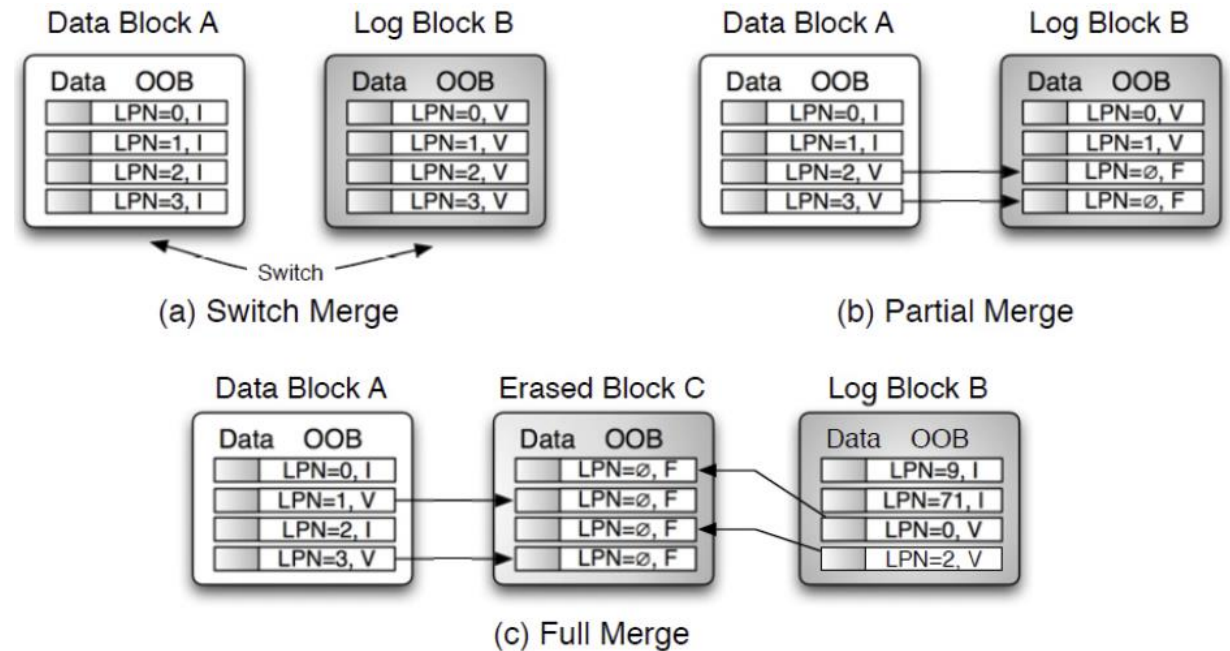
- Log block이 해당 하는 LBN의 모든 page를 valid로 보유했을 때, 맵핑만 바꾸는 방식

- 1) Partial Merge

- log block의 일부 page만 valid하고 log block의 빈 슬롯이 남아 있는 경우, log block에 data block의 나머지 valid page를 복사하는 방식

- 2) Full Merge

- Log block의 일부 page만 valid하고 log block의 빈 슬롯이 없을 경우(BAST), 혹은 여러 log block에 valid page가 분산되어 있을 때(FAST), 새로운 data block을 할당하여 모두 복사하는 방식



Ref: Gupta, Aayush, Youngjae Kim, and Bhuvan Uргаonkar. "DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings." *Acm Sigplan Notices* 44.3 (2009): 229-240.

1. Address Translation

- Merge : Switch
 - ✓ Write Requests

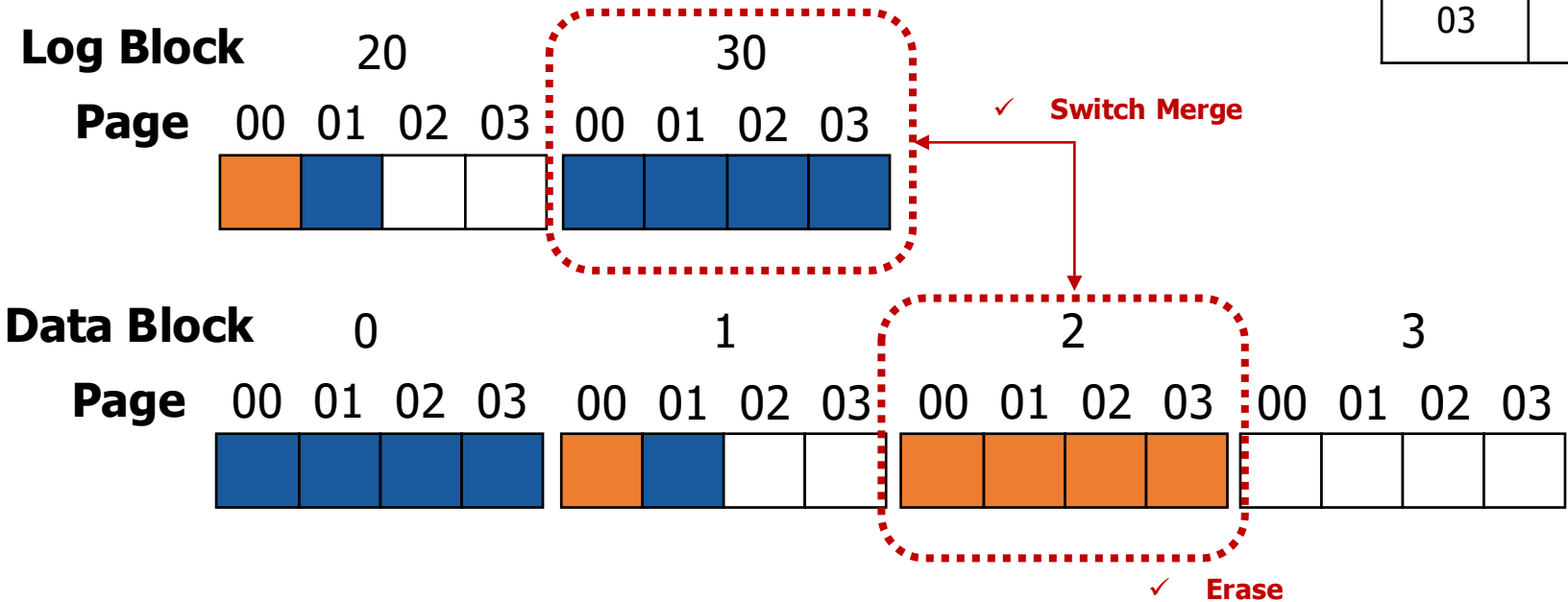
write(4, ...)	write(8, ...)	write(8, ...)
write(4, ...)	write(9, ...)	write(9, ...)
write(5, ...)	write(10, ...)	write(10, ...)
write(4, ...)	write(11, ...)	write(11, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	2→30
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,4}
02	30	{8,9,10,11}



1. Address Translation

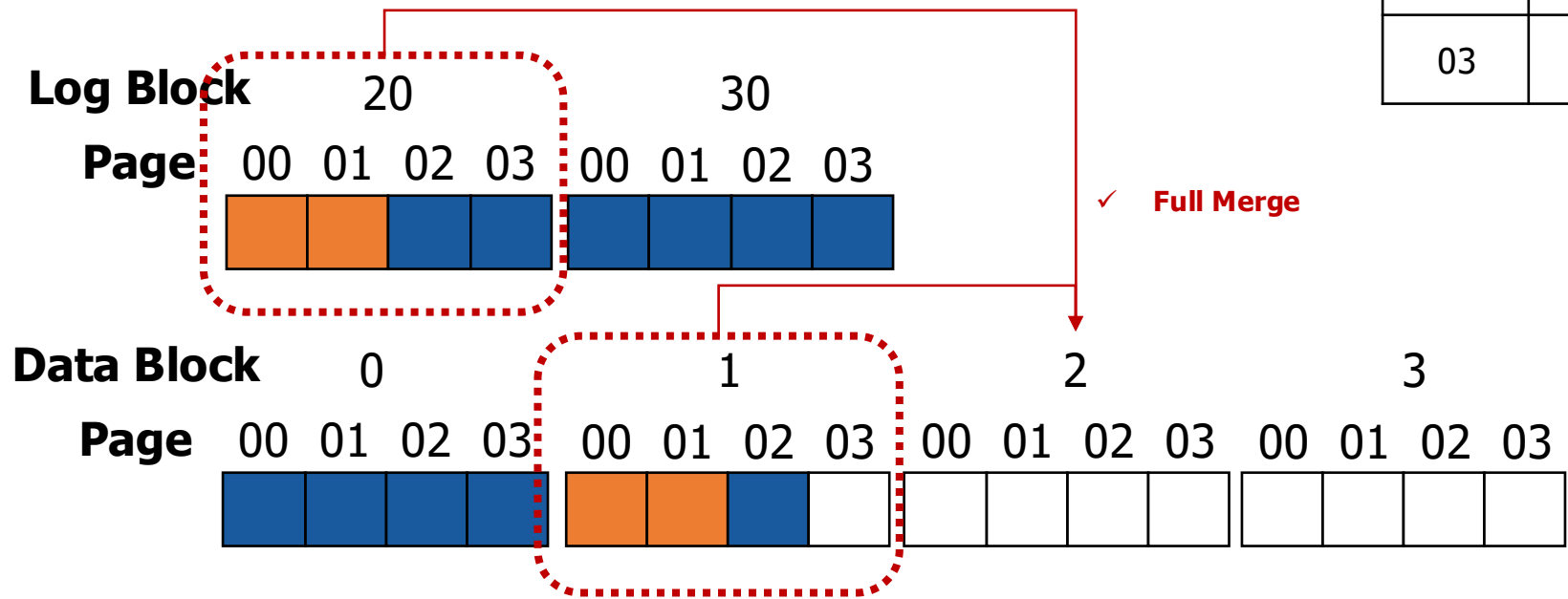
- Merge : Full
 - ✓ Write Requests
 - write(6, ...)
 - write(4, ...)
 - write(5, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	2→30
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,4,5,4}



1. Address Translation

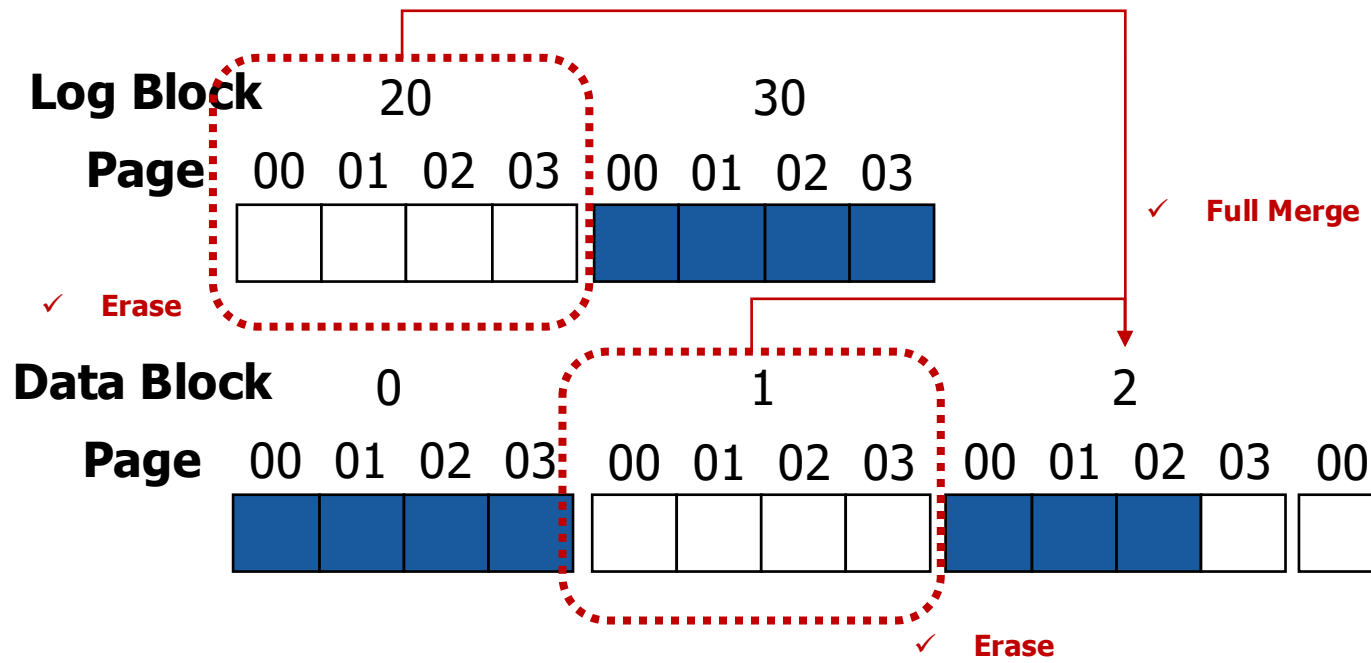
- Merge : Full
 - ✓ Write Requests
 - write(6, ...)
 - write(4, ...)
 - write(5, ...)

Block Mapping Table

LBN	PBN
00	0
01	1→2
02	2→30
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,4,5,4}



1. Address Translation

- Merge : Partial
 - ✓ Write Requests

write(4, ...)

write(6, ...)

write(4, ...)

write(7, ...)

write(5, ...)

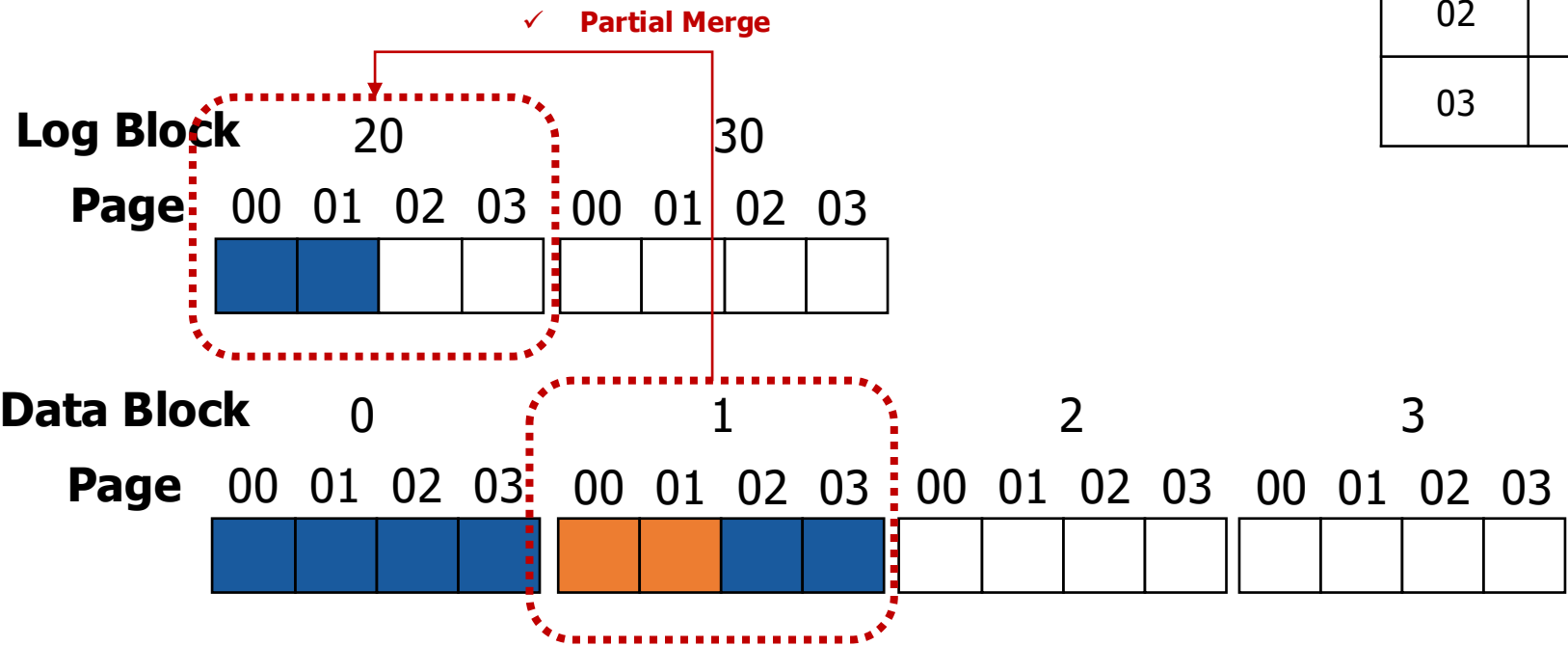
write(5, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,5}

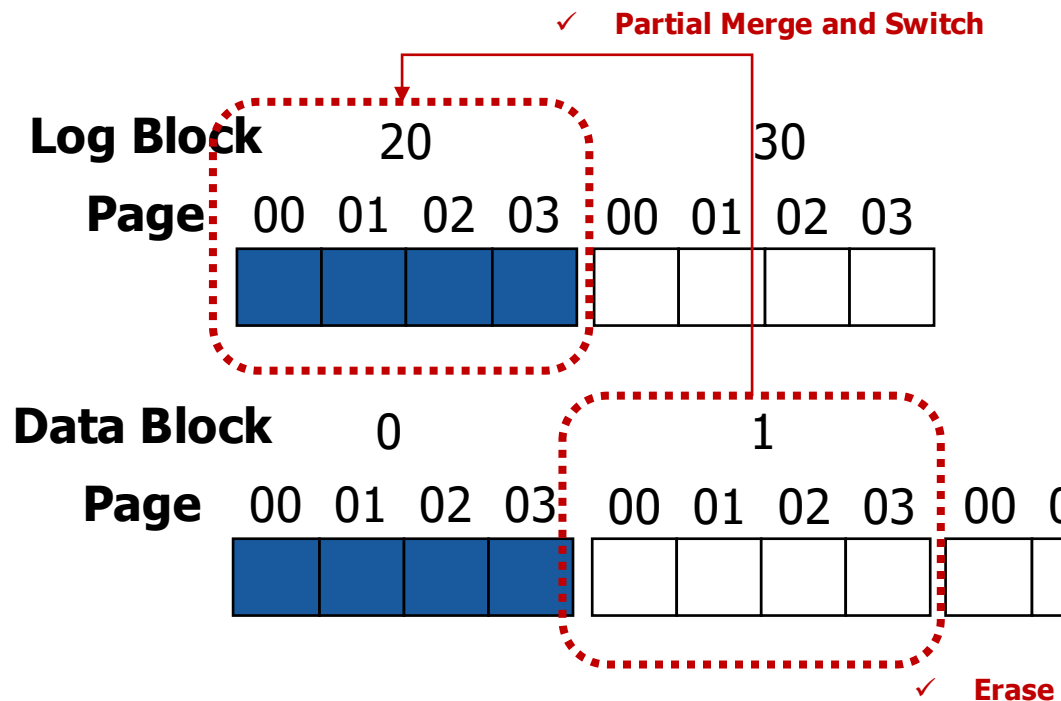


1. Address Translation

- Merge : Partial

- ✓ Write Requests

write(4, ...)	write(6, ...)
write(4, ...)	write(7, ...)
write(5, ...)	
write(5, ...)	



Block Mapping Table

LBN	PBN
00	0
01	1 → 20
02	
03	

Page Mapping Table for Log

LBN	PBN	Page
01	20	{4,5}

1. Address Translation

- **BAST algorithm (Block-Associative Sector Translation)**
 - ✓ 기본적으로 block mapping 구조를 활용하여 테이블 크기를 작게 유지
 - ✓ Log block을 활용하여 block내 일부 page 갱신 시에 활용
 - ✓ Log block과 Data block을 1:1로 대응하여 유지
 - ✓ 장점 : 작은 table, 구현 단순, 부분 업데이트 가능
 - ✓ 단점 : Log block의 1개 제약 → 특정 LBN이 많이 요청되면 merge가 빈번하게 발생

1. Address Translation

- FAST algorithm (Fully-Associative Sector Translation)

✓ Write Requests

write(4, ...)

write(8, ...)

write(5, ...)

write(12, ...)

write(4, ...)

write(8, ...)

write(5, ...)

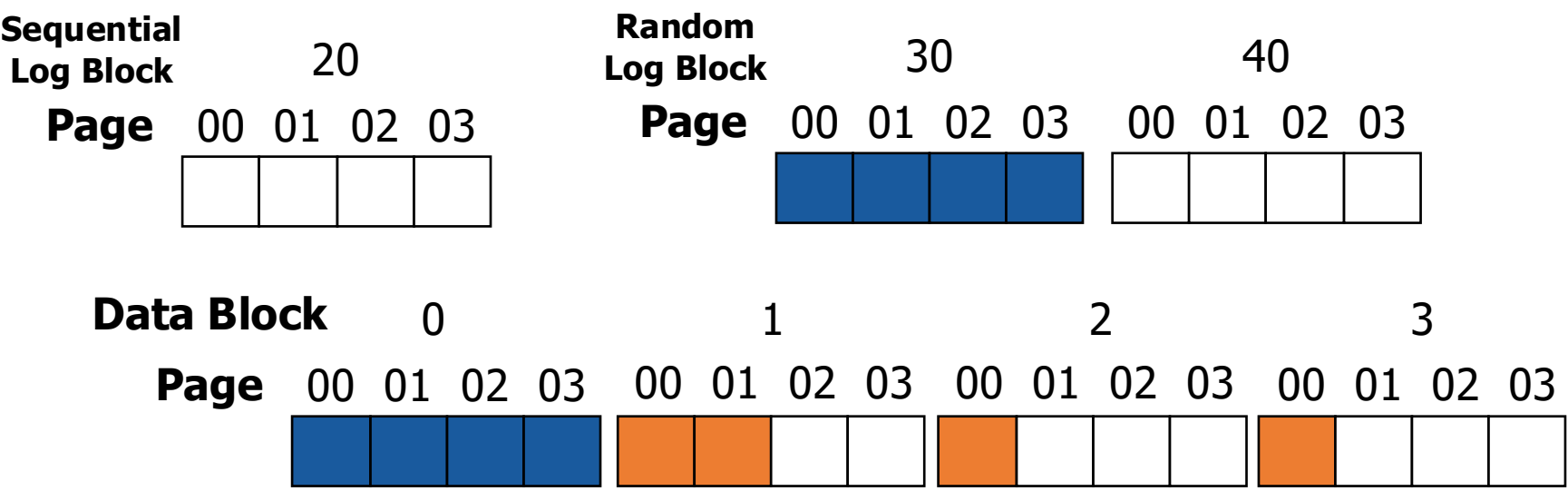
write(12, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	2
03	3

Page Mapping Table for Log

LBN	LPN	PBN	Offset
1	4	30	0
2	8	30	1
1	5	30	2
3	12	30	3



1. Address Translation

- FAST algorithm (Fully-Associative Sector Translation)

✓ Write Requests

write(4, ...)

write(5, ...)

write(6, ...)

write(7, ...)

write(4, ...)

write(5, ...)

write(6, ...)

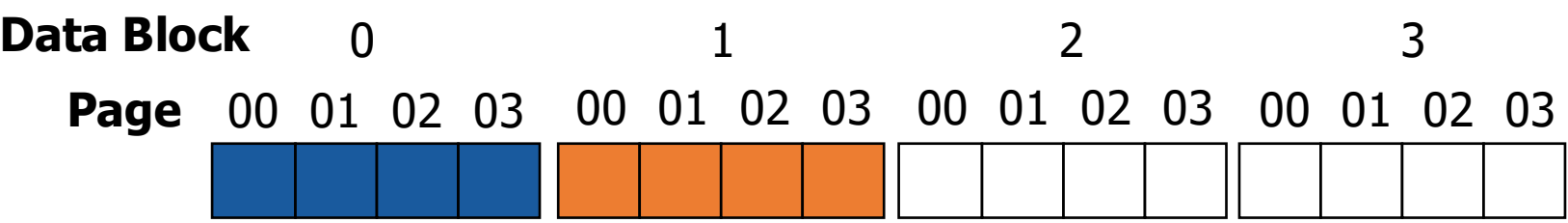
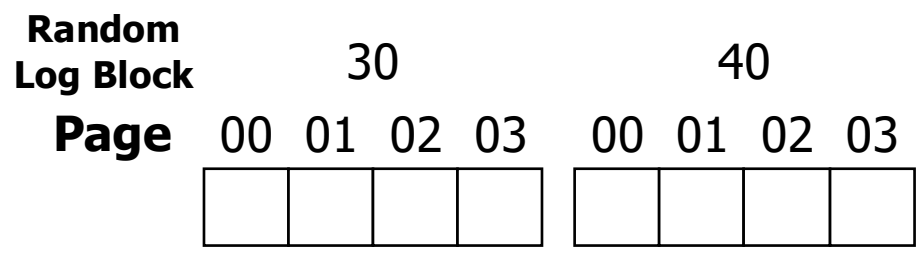
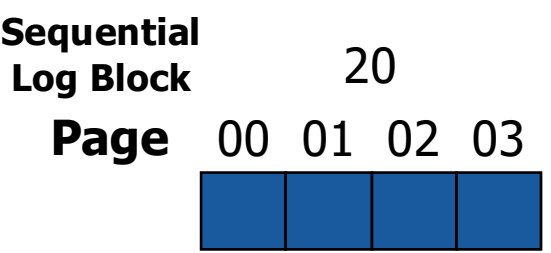
write(7, ...)

Block Mapping Table

LBN	PBN
00	0
01	1

Page Mapping Table for Log

LBN	LPN	PBN	Offset
1	4	20	0
1	5	20	1
1	6	20	2
1	7	200	3



Valid page Invalid page Erased page (Free page)

1. Address Translation

- FAST algorithm (Fully-Associative Sector Translation)

✓ Write Requests

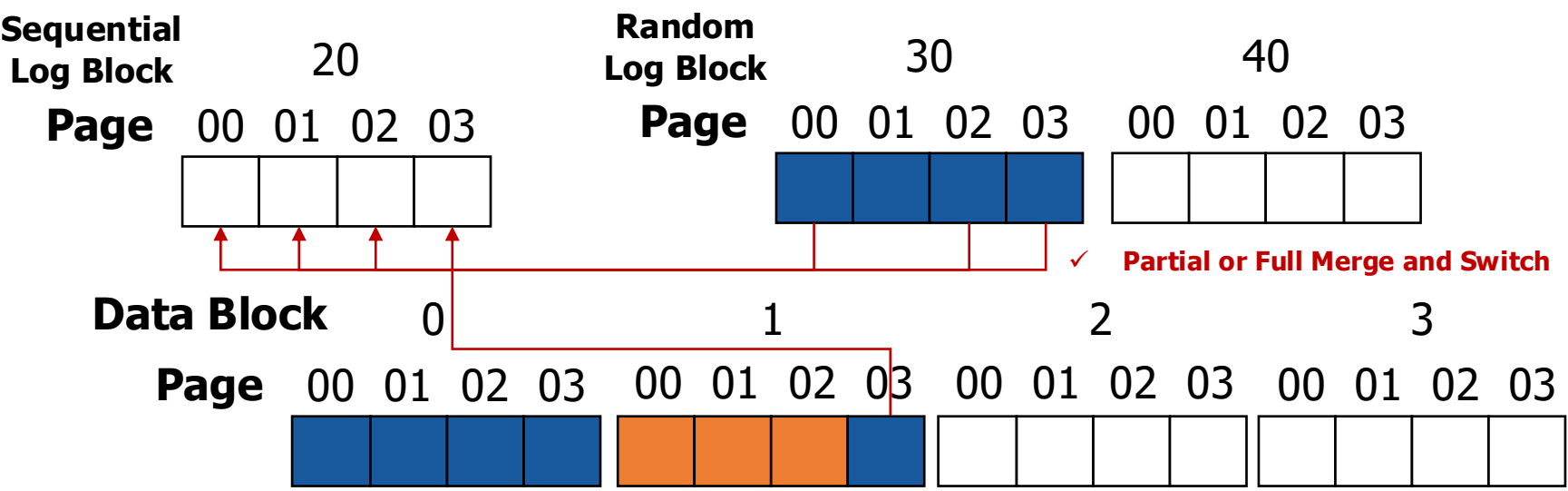
write(4, ...)
write(9, ...)
write(5, ...)
write(6, ...)

Block Mapping Table

LBN	PBN
00	0
01	1
02	
03	

Page Mapping Table for Log

LBN	LPN	PBN	Offset
1	4	30	0
2	9	30	1
1	5	30	2
1	6	30	3



1. Address Translation

- **FAST algorithm (Fully-Associative Sector Translation)**
 - ✓ 기본적으로 block mapping 구조를 활용하여 테이블 크기를 작게 유지
 - ✓ Log block을 활용하여 block내 일부 page 갱신 시에 활용
 - ✓ LBN당 Log block 1개 제약이 아니라, 모든 LBN이 Global Log Block Pool을 사용
 - 특정 LBN의 반복된 요청으로 인한 merge 완화
 - ✓ 장점 : BAST의 단점 극복 → 쓰기 증폭 줄임
 - ✓ 단점 : 메타데이터의 크기가 증가, merge 정책의 구현 난이도 증가

2. Wear-Leveling

- Hot/Cold Data
 - Hot data
 - ✓ Update 빈도가 높은 데이터
 - ✓ 동일한 위치에 반복적으로 쓰기 요청이 발생하는 데이터
 - ✓ 해당 block 안의 hot 데이터는 시간이 지나면서 대부분 무효화(Invalid) 상태가 됨
 - ✓ 주로 로그 데이터, 캐시 데이터, 메타 데이터 같은 영역에 많이 발생
 - Cold data
 - ✓ Update 빈도가 매우 낮거나 거의 없는 데이터
 - ✓ 한번 기록되면 장기간 수정되지 않는 데이터
 - ✓ 블록 내 cold 데이터는 대부분 여전히 유효(Valid) 상태로 남음
 - ✓ 주로 시스템 파일, 실행 프로그램 코드 등이 있음

2. Wear-Leveling

- **Wear-leveling**

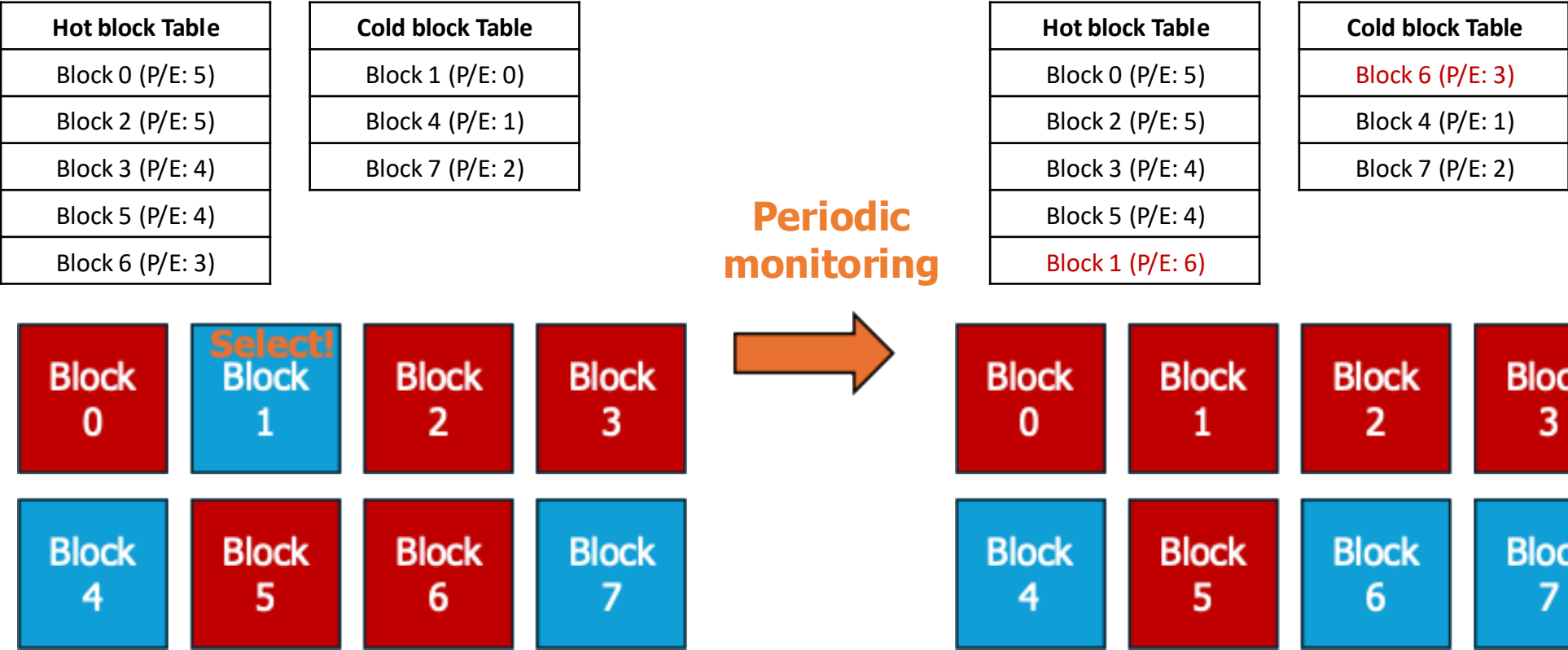
- ✓ Block 단위 모니터링 (monitoring)
- ✓ P/E cycle을 모든 block에 고르게 분산시켜 특정 block의 과도한 사용을 방지하고, 장치의 전체 수명을 연장하는 기술

- *ECN(Erase Counter Number) based wear-leveling algorithms*
- *Write based wear-leveling algorithms*
- *Group based wear-leveling algorithms*

2. Wear-Leveling

- ECN based wear-leveling algorithms

- Block 별 erase 연산 횟수를 기반으로 모니터링하여 과다 마모 block의 데이터를 덜 마모된 block으로 재배치함
-
- 1) 각 block의 ECN 값과 그 평균을 계산
 - 2) 평균 보다 크면 hot block, 작으면 cold block으로 구분
 - 3) 혹은 특정 값을 기준으로 할 수 있음

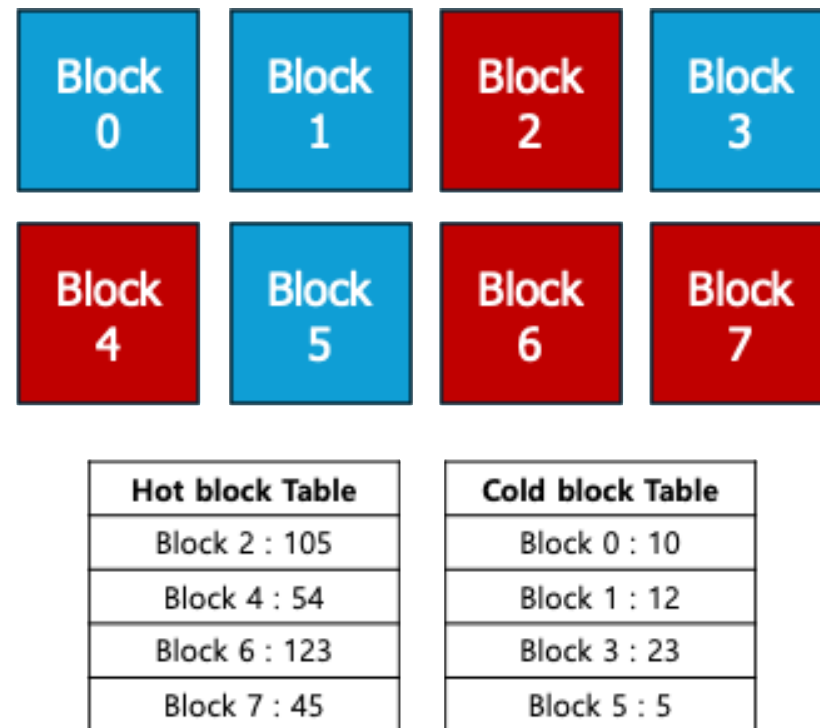
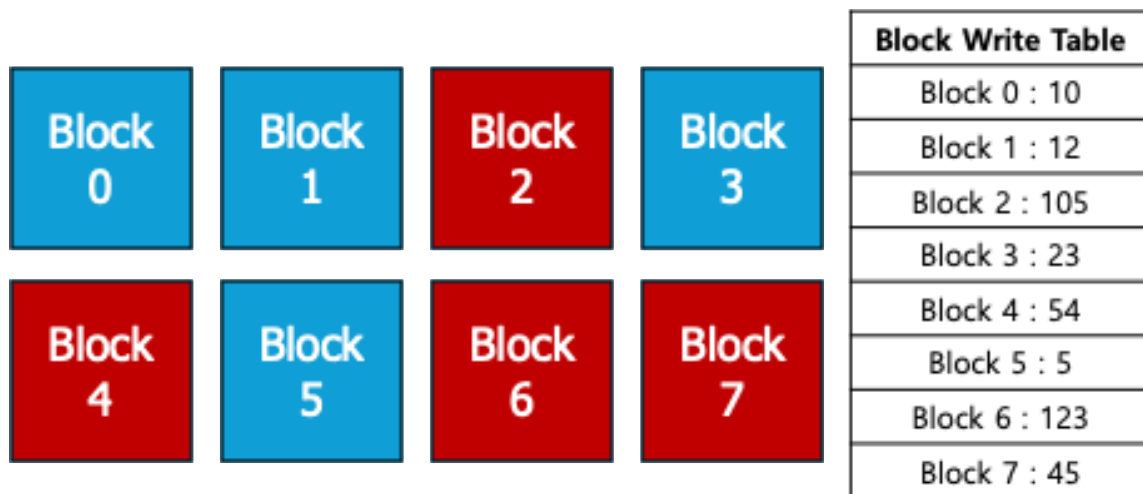


2. Wear-Leveling

- Write based wear-leveling algorithms

✓ Block 별 write 연산 횟수를 기반으로 모니터링하여 hot data를 덜 마모된 block으로 우선 배치하여 단기 마모 집중 완화

- 1) Write count per block
- 2) Hot/Cold table

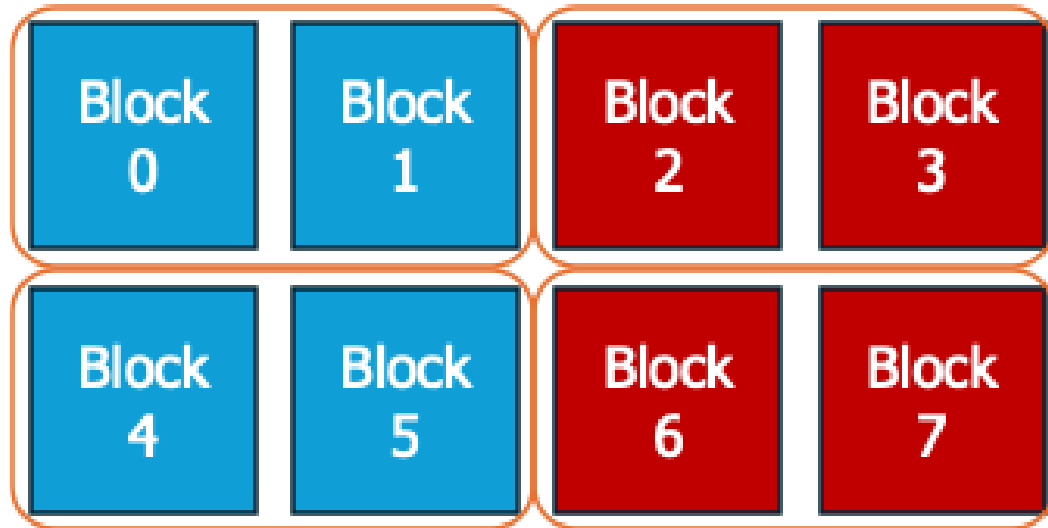


2. Wear-Leveling

- Group based wear-leveling algorithms

- ✓ 여러 block을 큰 단위(zone, bank)로 묶어서 평균 지표(ECN/write)를 기준으로 그룹 간 스와핑을 통하여 마모 관리

- 1) Zone/Bank 단위로 묶음 (4~20개, 혹은 그 이상)
- 2) 각 zone/bank의 모든 block의 ECN이나 write count의 평균을 계산



Zone Write Table
Zone 0 : 22
Zone 1 : 128
Zone 2 : 59
Zone 3 : 168

Block Write Table
Block 0 : 10
Block 1 : 12
Block 2 : 105
Block 3 : 23
Block 4 : 54
Block 5 : 5
Block 6 : 123
Block 7 : 45

2. Wear-Leveling

- ECN, Write, Group 기반 알고리즘 비교

구분	ECN based	Write based	Group based
지표	Block별 ECN	Block별 Write count	여러 블록의 ECN, WC
목표	ECN 편차 줄여 장기적으로 마모 균등화	hot write로 인한 단기 마모 완화	메타데이터 절감 및 큰 단위의 균등화
장점	단순함 안정적임	워크로드 변화에 민첩함	RAM 사용량을 크게 줄임
단점	단기 hot block 대응 미미	매 연산에 따른 연산 오버헤드	세밀 제어 한계 (그룹 내 불균형)

3. Garbage Collection

- **Garbage Collection (GC, Cleaning)**

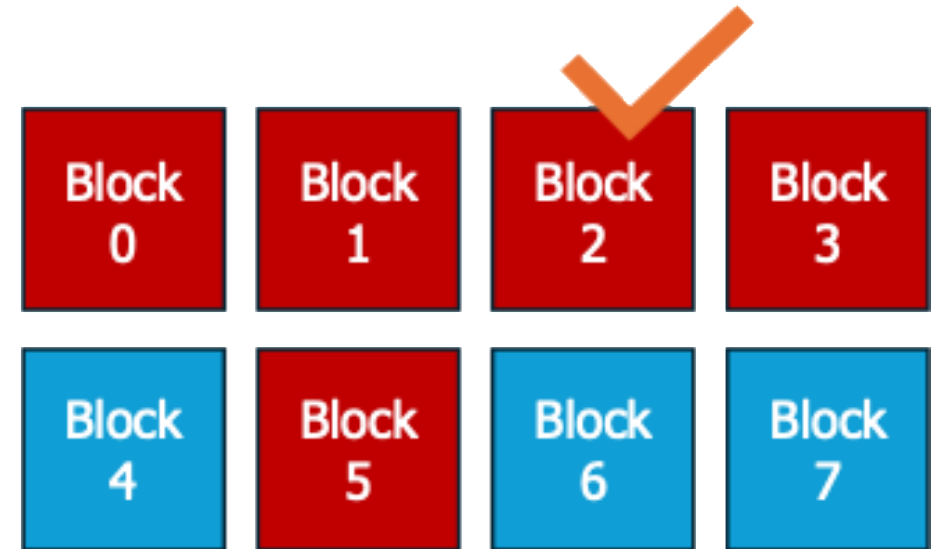
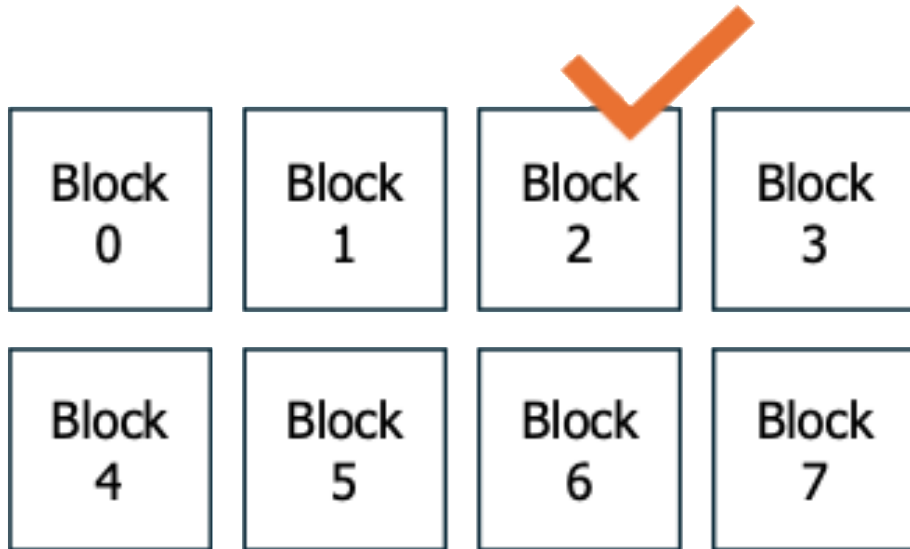
- ✓ Page 단위 모니터링 (monitoring)
- ✓ FTL에서 무효 (invalid) page가 많은 블록을 골라, valid 데이터만 다른 block으로 복사한 뒤 block을 erase하여 공간을 회수하는 기법

- *Random*
- *Greedy*
- *Cost-Benefit*
- *CAT (Cost-Age-Time)*

3. Garbage Collection

- Random algorithms

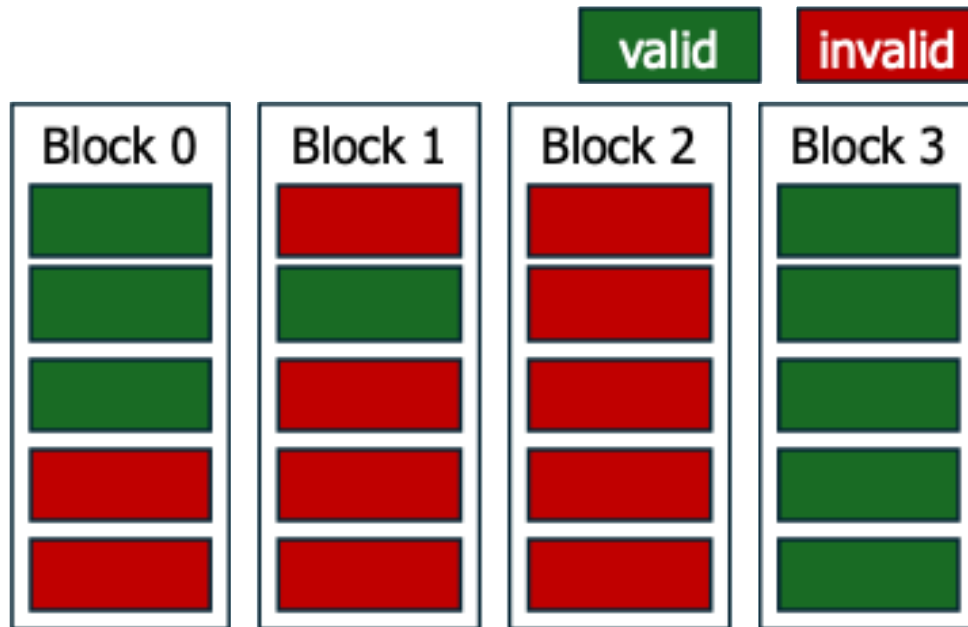
- ✓ 후보 block들 중에서 무작위로 희생(victim) block 선택
 - 메타데이터, 계산 오버헤드가 가장 적지만, 효율과 마모 균등화가 불안정



3. Garbage Collection

- Greedy algorithms

- ✓ 무효(invalid) page 비율(=유효 비율 u 가 가장 작은 block)이 가장 큰 block을 선택
 - 즉시 최대 여유 공간을 회수하는데 효과적, 마모 균등화에서 불리할 수 있음



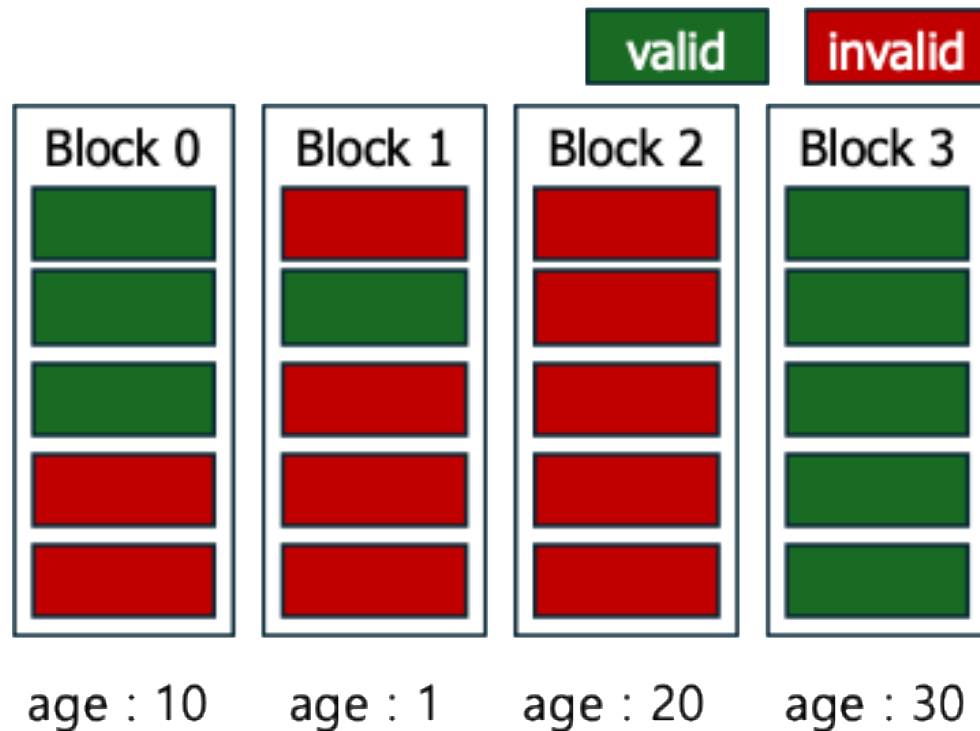
$$u = \frac{\# \text{ valid pages in block}}{\# \text{ total pages in block}}$$

- Block 0 : $3/5 = 0.6$
- Block 1 : $1/5 = 0.2$
- Block 2 : $0/5 = 0.0$
- Block 3 : $5/5 = 1.0$

3. Garbage Collection

- Cost-Benefit algorithms

- ✓ 아래 식을 계산한 점수가 **최대**인 block 선택
 - 회수 이득과 비용, 오래됨을 함께 고려하는 균형 정책
 - age의 경우, 상한이 존재



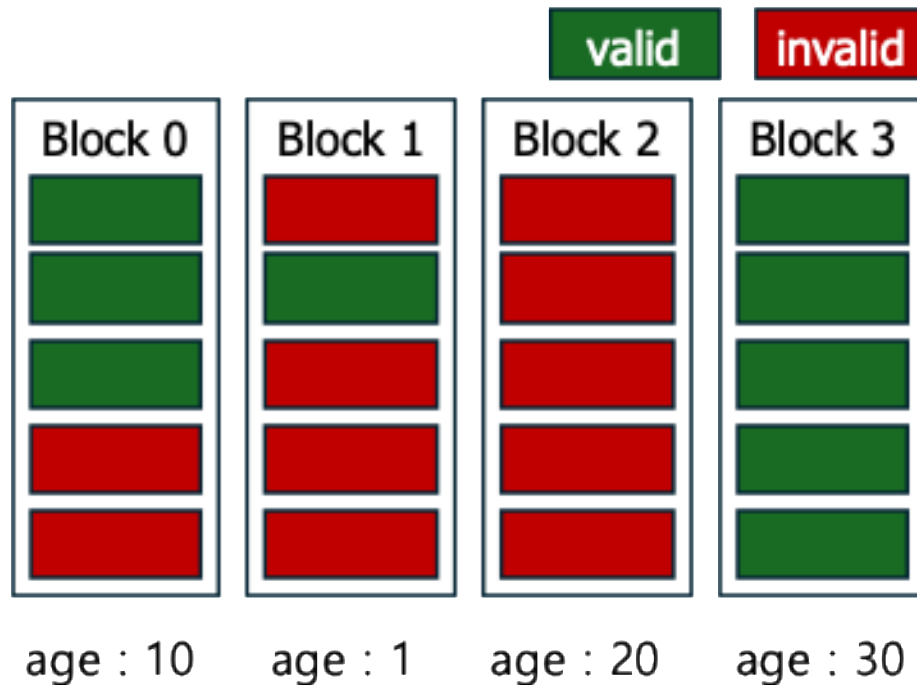
$$cleaning = age * \frac{(1 - u)}{2u}$$

- u : 유효 데이터 비율
- $(1 - u)$: 회수 가능 공간 비율
- age : 최근 수정 이후 경과 시간
- $2u$: $read(u) + write(u)$

3. Garbage Collection

- CAT(Cost-Age-Time) algorithms

- ✓ 아래 식을 계산한 점수가 **최소**인 block 선택
 - 단기적인 hot block을 줄임으로써 마모를 완화



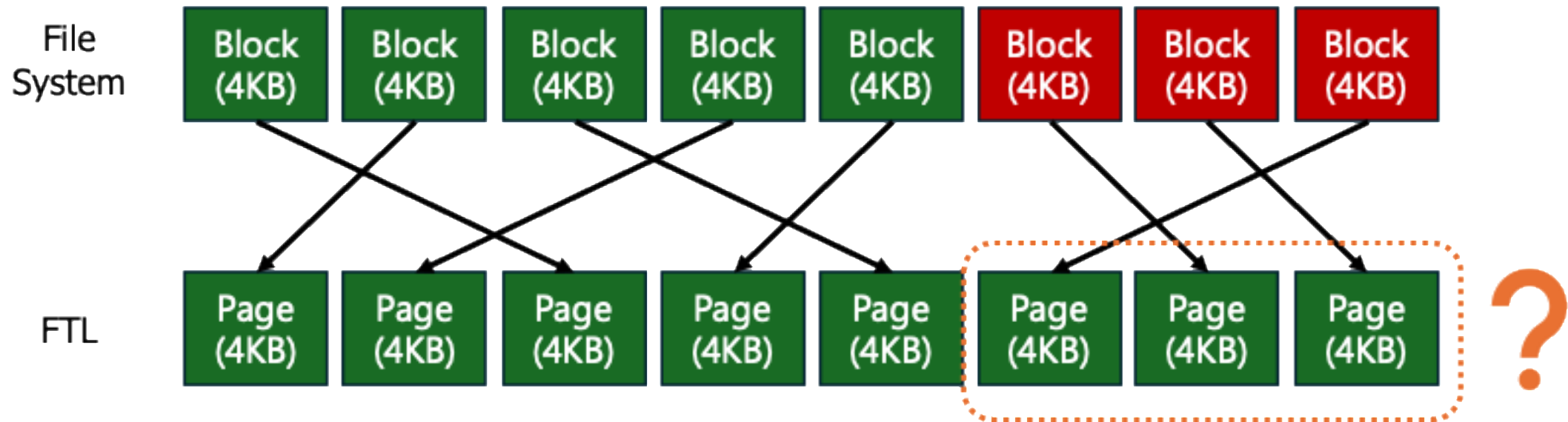
$$cleaning = \frac{u}{1 - u} * \frac{1}{age} * number_of_cleaning$$

- u : 유효 데이터 비율
- $(1 - u)$: 회수 가능 공간 비율
- age : 최근 수정 이후 경과 시간
- $number_of_cleaning$: erase count

4. TRIM

- TRIM

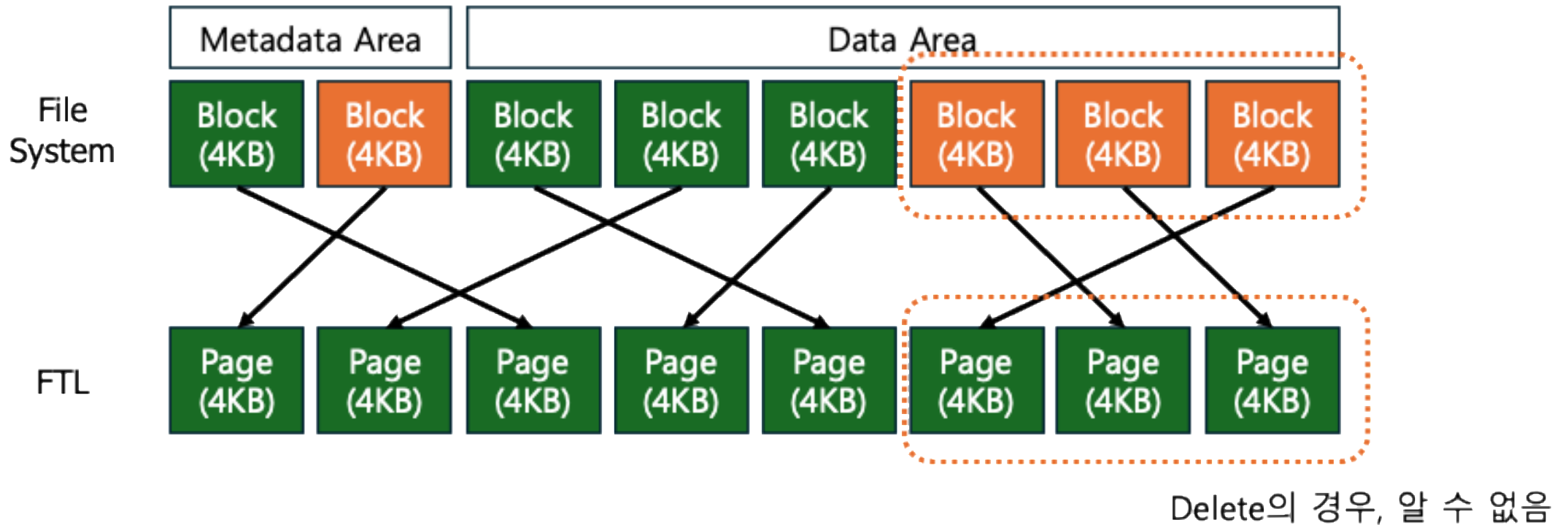
- ✓ 파일시스템과 같은 SSD에 요청하는 작업자가 삭제된 LBA 범위를 SSD에게 알려, FTL이 해당 매핑을 모두 무효(invalid) 상태로 표기하도록 만드는 힌트(hint) 명령



4. TRIM

- TRIM

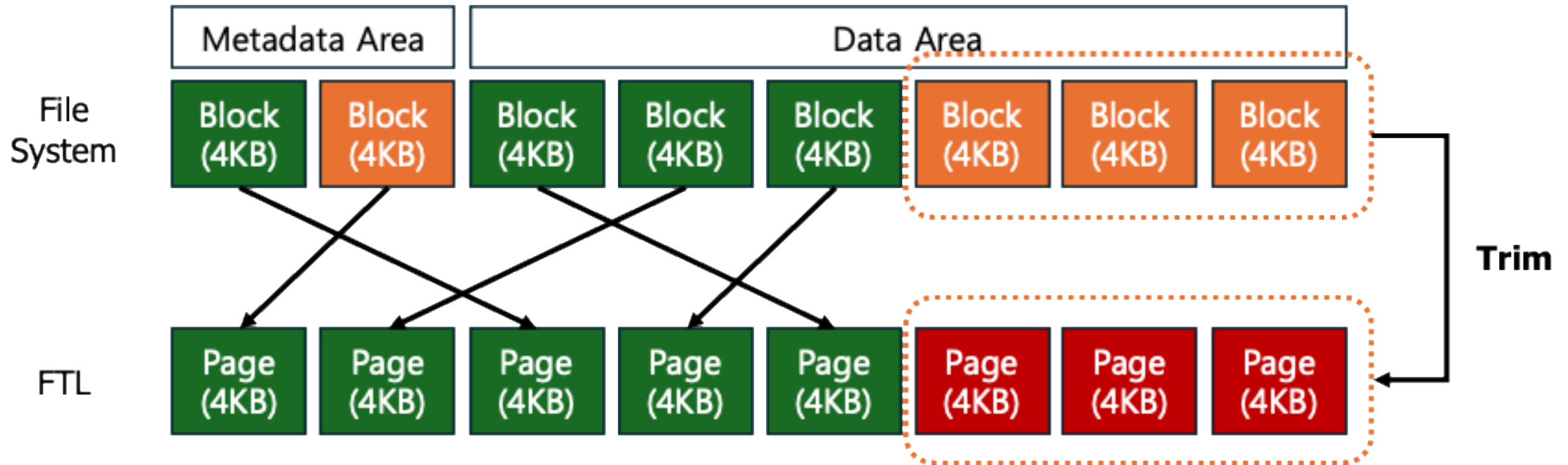
- ✓ 파일시스템과 같은 SSD에 요청하는 작업자가 삭제된 LBA 범위를 SSD에게 알려, FTL이 해당 매핑을 모두 무효(invalid) 상태로 표기하도록 만드는 힌트(hint) 명령



4. TRIM

- TRIM

- ✓ 파일시스템과 같은 SSD에 요청하는 작업자가 삭제된 LBA 범위를 SSD에게 알려, FTL이 해당 매핑을 모두 무효(invalid) 상태로 표기하도록 만드는 힌트(hint) 명령



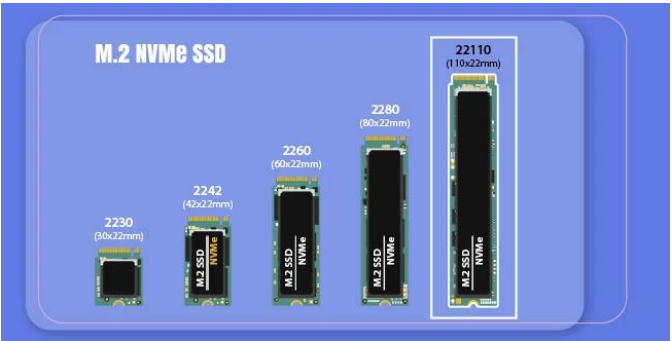
5. ETC

- Form Factor

Client



M.2

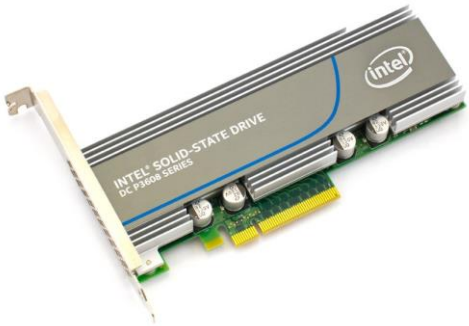
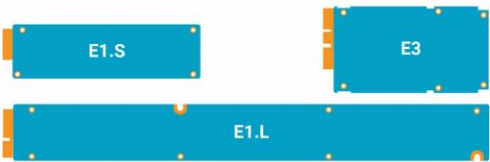


mSATA



Figure 4: EDSFF family

Server



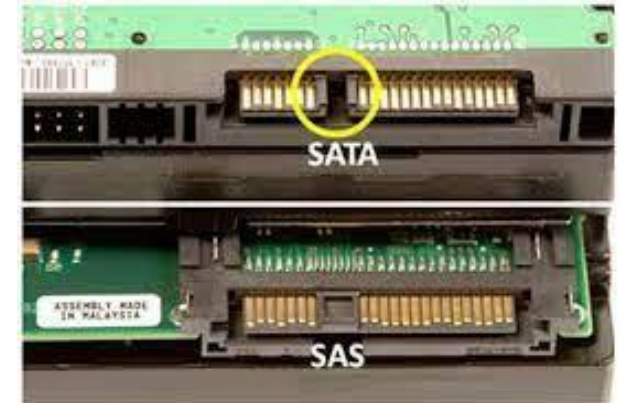
U.2 / U.3

E1.S / E1.L / E3

AIC
(Add-In-Card)

5. ETC

- Interface
- SATA
 - Protocol : AHCI (Advanced Host Controller Interface)
 - 구현 인터페이스이나 호환성이 매우 높고 보급형 시스템에서 널리 사용됨.
- SAS
 - Protocol : SCSI (Small Computer System Interface)
 - 서버/엔터프라이즈와 같은 고신뢰를 요구하는 환경에서 사용하고, 핫플러그를 지원함.
- PCIe
 - Protocol : NVMe (Non-Volatile Memory Express)
 - SSD를 위해 설계된 고성능, 낮은 지연, 병렬처리를 지원하는 인터페이스



5. ETC

- Interface
- PCIe
 - Protocol : NVMe (Non-Volatile Memory Express)
 - SSD를 위해 설계된 고성능, 낮은 지연, 병렬처리를 지원하는 인터페이스

Revision	Max Data Rate	Encoding	Signaling
PCIe 7.0 (2025)	128.0 GT/s	1b/1b (Flit Mode*)	PAM4
PCIe 6.0 (2022)	64.0 GT/s	1b/1b (Flit Mode*)	PAM4
PCIe 5.0 (2019)	32.0 GT/s	128b/130b	NRZ
PCIe 4.0 (2017)	16.0 GT/s	128b/130b	NRZ
PCIe 3.0 (2010)	8.0 GT/s	128b/130b	NRZ
PCIe 2.0 (2007)	5.0 GT/s	8b/10b	NRZ
PCIe 1.0 (2003)	2.5 GT/s	8b/10b	NRZ

(*Flit Mode also enabled in other Data Rate with their respective encoding)

급발전하는 PCIe, 이젠 'PCIe 7.0' 넘어 '8.0'...

전문기자 | 입력 2025.06.16 17:31 | 댓글 1



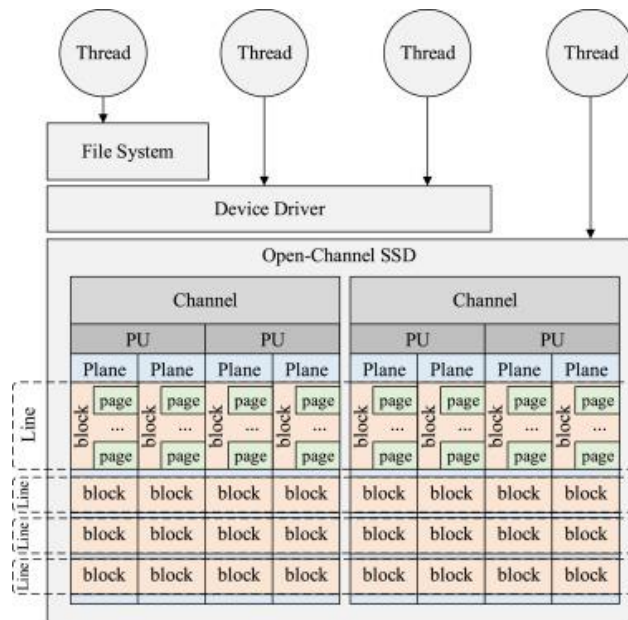
최근 PCI SG 컨소시엄 '7.0' 인증, 512GBps 속도 등 '사양 공개'
PCIe 6.0보다 양방향 대역폭 2배, 하위 호환성, 효율성 향상
소비자용 하드웨어에 적용은 5년 후쯤... "PCI 8.0도 본격 개발 중"



PCIe 7.0 슬롯이 장착된 마더보드. (출처=PCmag)

5. ETC

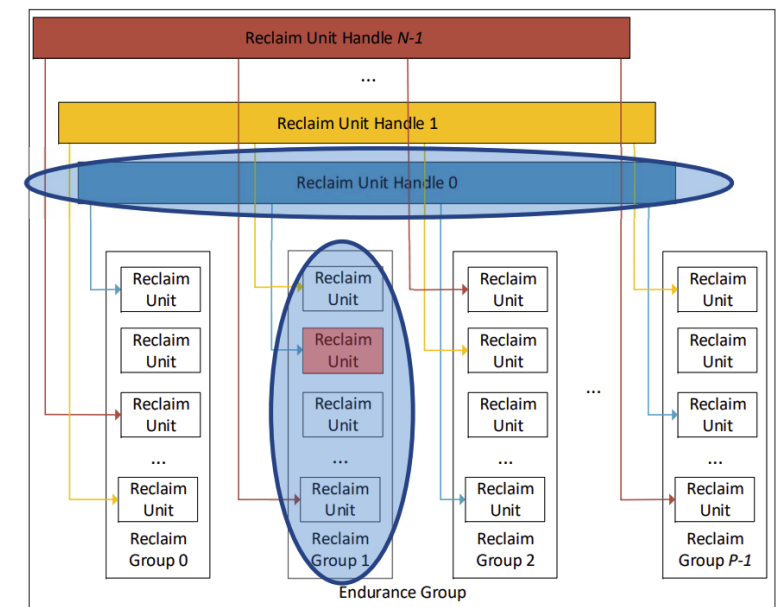
- 호스트-협업형 SSD (Host-cooperative SSD)
 - 호스트가 쓰기, 배치 등의 힌트로 SSD 내부 동작을 제어 혹은 협업하여 최적하는 계열의 SSD



Open-Channel SSD



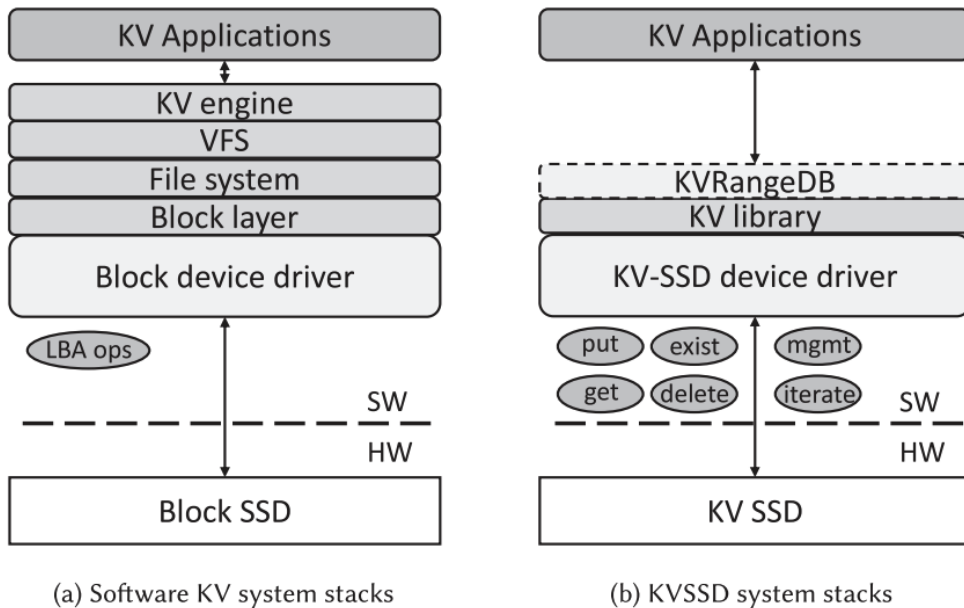
ZNS SSD
(Zoned Namespace SSD)



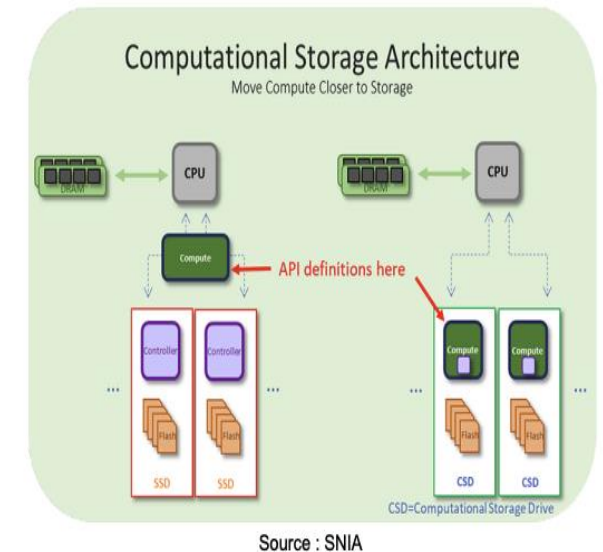
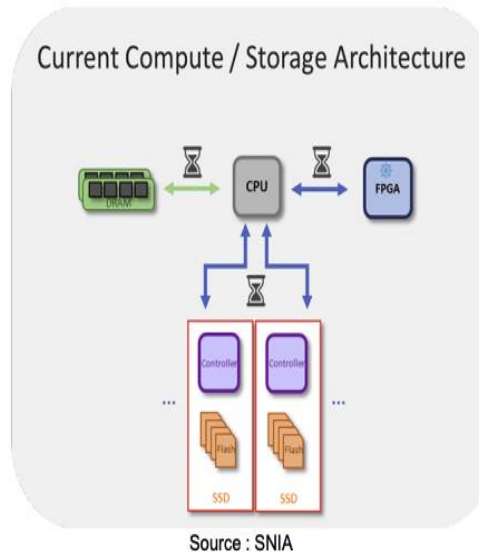
FDP
(Flexible Data Placement)

5. ETC

- 기능-확장형 SSD (Function-offloaded SSD)
 - 저장 장치가 호스트 어플리케이션의 기능 일부를 직접 수행하여 호스트 부담을 줄이는 계열의 SSD



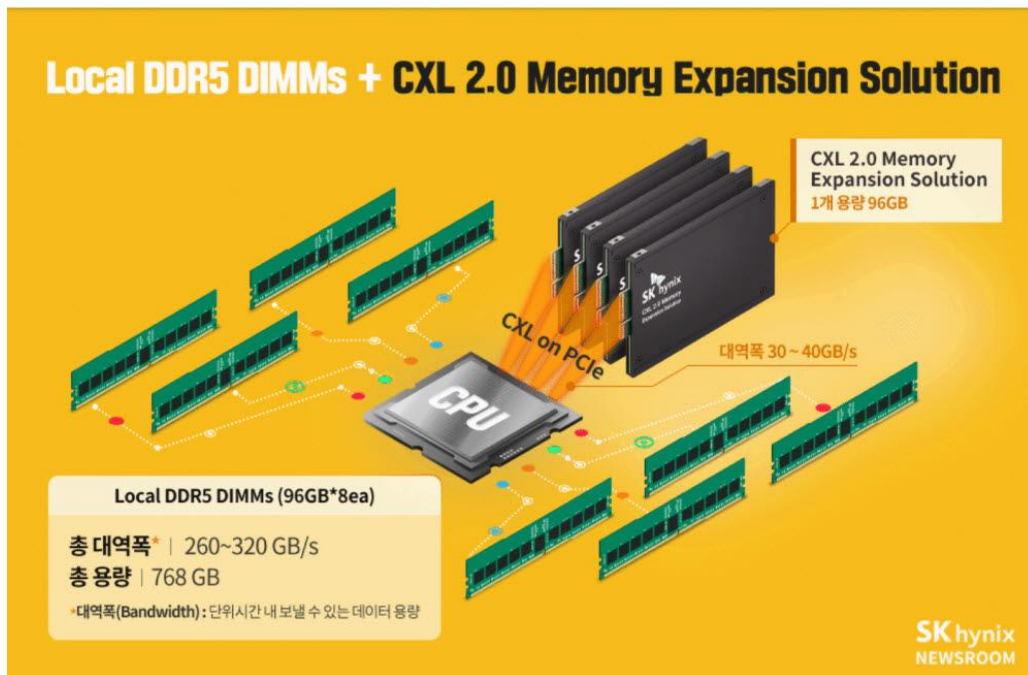
KV SSD
(Key-Value SSD)



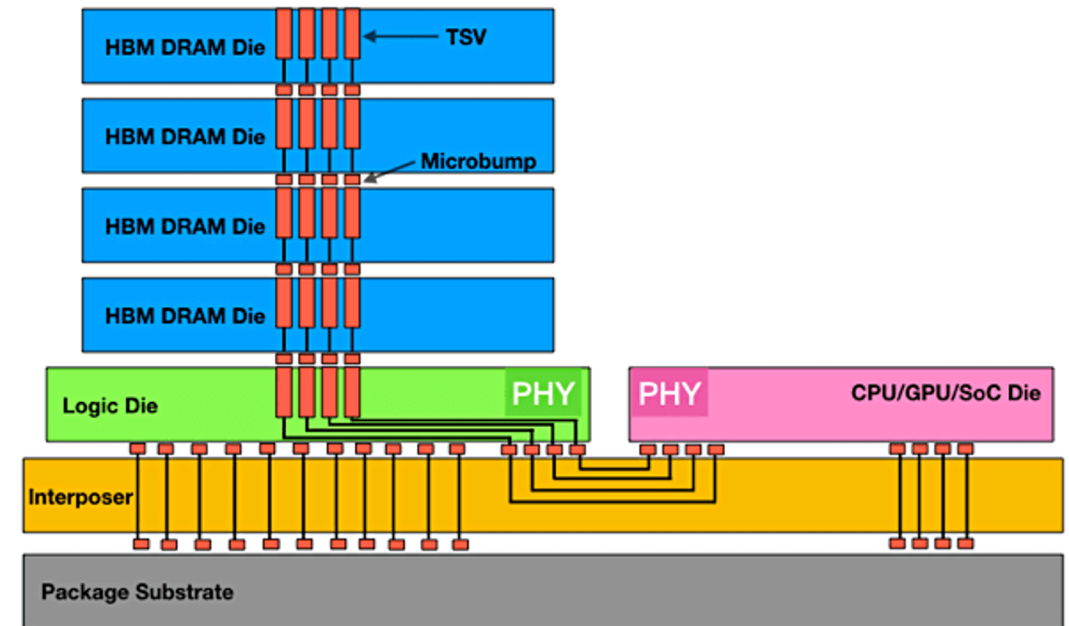
CSD
(Computational Storage SSD)

5. ETC

- 메모리-스토리지 융합 SSD
 - 고대역폭 메모리와 풀링/확장을 이용하여 스토리지-메모리 경계를 좁히는 계열의 SSD



CXL
(Compute Express Link)



HBM
(High Bandwidth Memory)

6. Research Topic

- File System
 - LFS(Log-Structured FS)
 - **Who?**
 - ✓ John Ousterhout and Mendel Rosenblum (from UC Berkeley)
 - **Motivations**
 - ✓ **System memories are growing**
 - More caching → most reads are serviced by the cache → main disk traffics are writes
 - ✓ A large gap between random and sequential writes
 - Modern disks: 1) high density, but 2) relatively slow improvements in positioning (seek and rotational) delay → random becomes serious bottleneck in disks
 - ✓ Existing FS generates random writes
 - For example, FFS incurs multiple small random writes for a file creation (inode, bitmap, directory block, ...) even though they are within the same block group → cause multiple positioning delays
 - ✓ FS needs RAID-awareness
 - RAID has small-write problem for parity → A write to a single block causes 4 physical I/Os

6. Research Topic

- File System
 - F2FS (Flash-friendly File System)

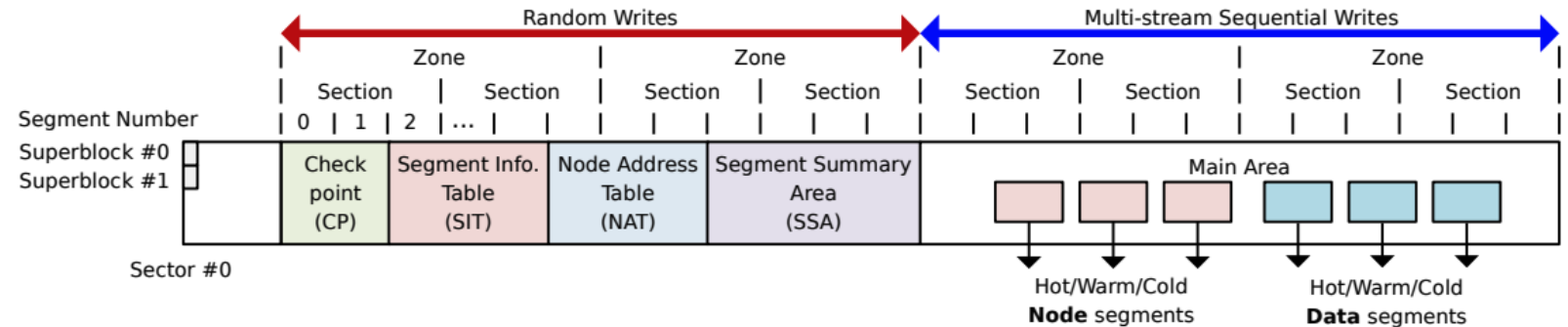
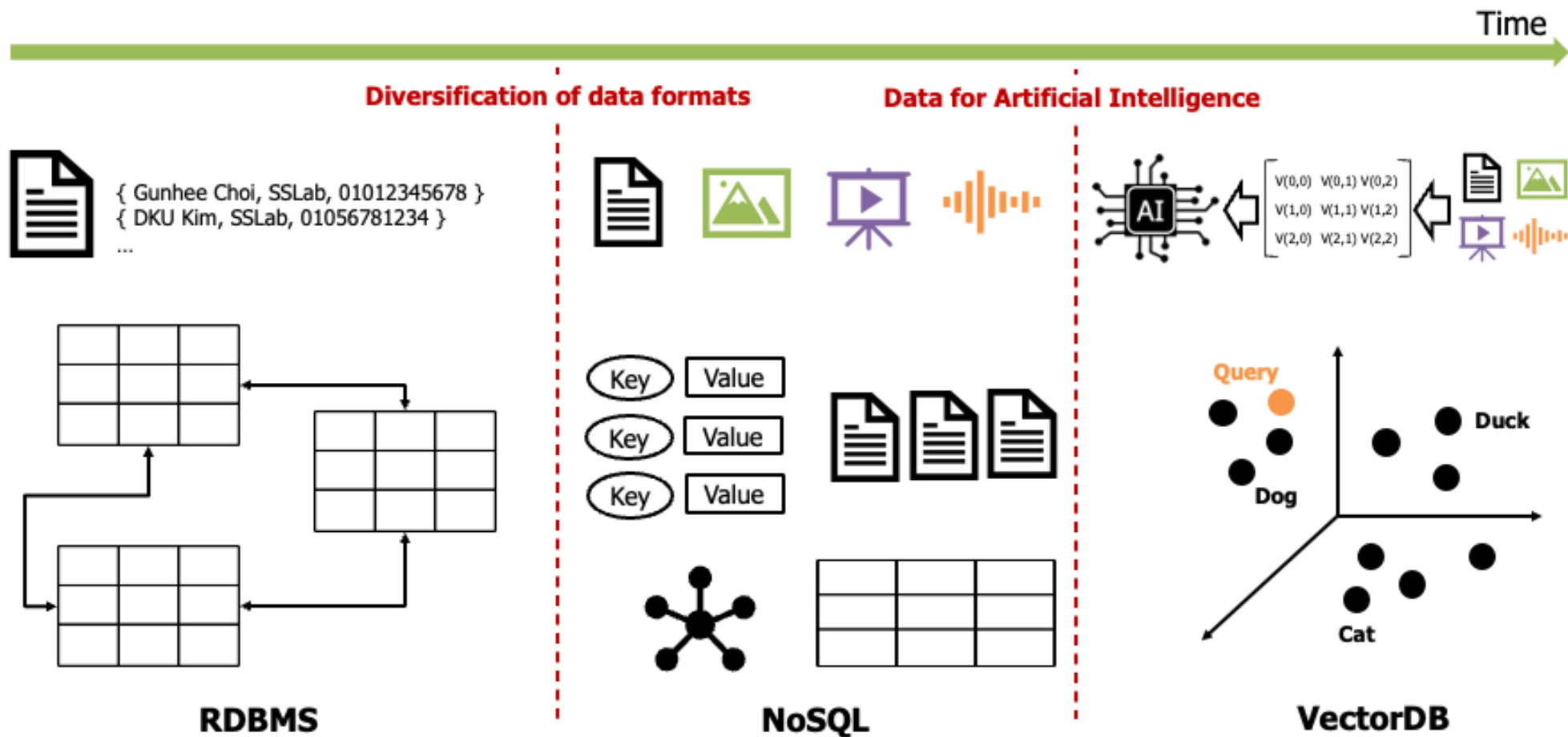


Figure 1: On-disk layout of F2FS.

- ✓ **Hot/Warm/Cold separation**
 - Write updates to separate logs by update frequency to reduce copy cost during GC, lowering WAF.
- ✓ **Segment/section/zone alignment**
 - Align writes with the device's erase-block boundaries to keep the FTL's internal operations in lockstep.
- ✓ **Live-block tracking via SIT/NAT/SSA**
 - Enables faster victim selection and background GC, mitigating tail latency.
- ✓ **Aggressive TRIM/Discard**
 - Promptly informs the FTL of invalidated blocks, improving internal GC efficiency.

6. Research Topic

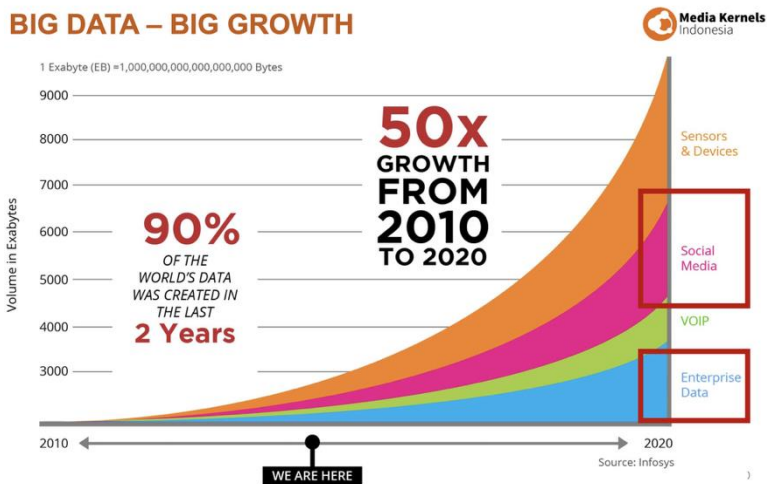
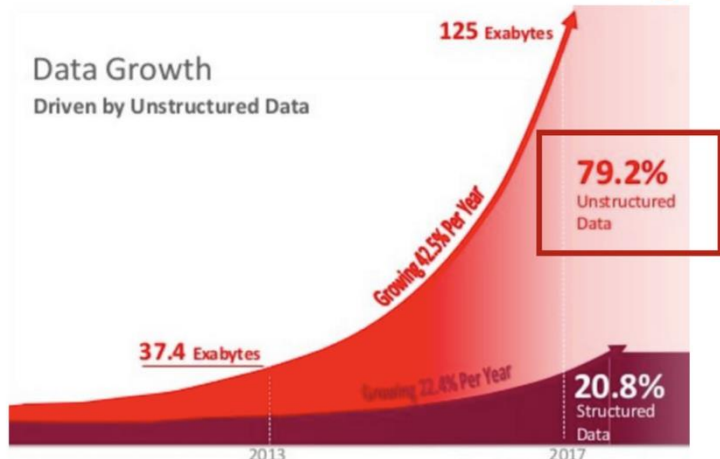
- Key-Value Store



6. Research Topic

- Key-Value Store : Unstructured Data

DATA GROWTH: UNSTRUCTURED DATA



Google

- Bigtable, Level DB, Hbase
- For Web indexing and messaging

Amazon

- Dynamo, SimpleDB
- For E-commerce

ORACLE

Oracle

- NoSQL, Berkeley DB
- For Configurable I

amazon.com

Microsoft

- Azure, Cosmos DB
- For E-commerce

Facebook

- Haystack, RocksDB, Cassandra
- For social network and photo store

facebook

Microsoft

Baidu

- Atlas
- For Cloud data

Basho

- Riak
- For distributed KV

YAHOO!

- PNUTS
- For Advertising

Yahoo

- Voldemort
- For Scalability

LinkedIn

- Voldemort
- For Scalability

Baidu

- Redis, Memcached
- For in-memory DB, cache

Open source

Redis, Memcached

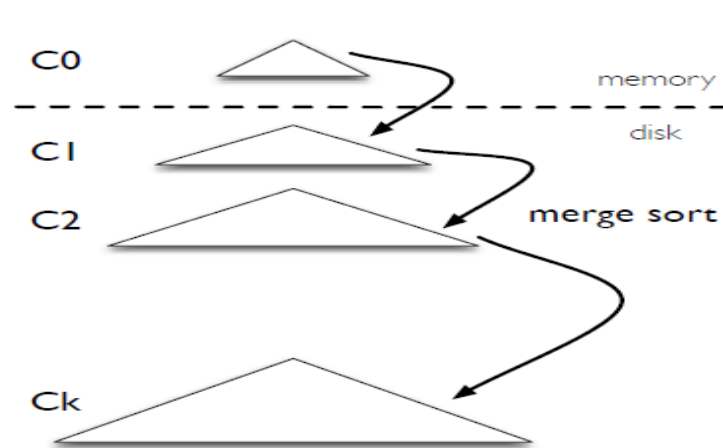
For in-memory DB, cache

Ref: Lee, Changman, et al. "{F2FS}: A new file system for flash storage." *13th USENIX Conference on File and Storage Technologies (FAST 15)*. 2015.

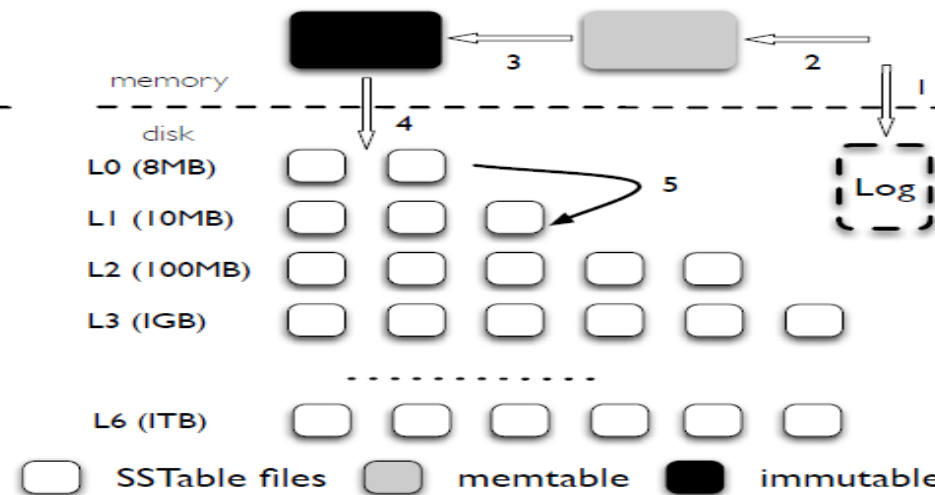
6. Research Topic

- LSM-tree (Log-Structured Merge Tree)

- Based on LSM (Log Structured Merge)-Tree
 - Write-optimized data structure
- Component: MemTable, SSTable, Log (WAL)
- Operations
 - User visible: put(key, value), get(key), iteration, ..
 - Internal
 - Flush (Dump): From MemTable to SSTable, append like log
 - Compaction (Merge): Size limit at each level, sorted, no overlap
 - Lookup (Indexing): Memtable first, From L0 to L6, Key fence + Bloom filter



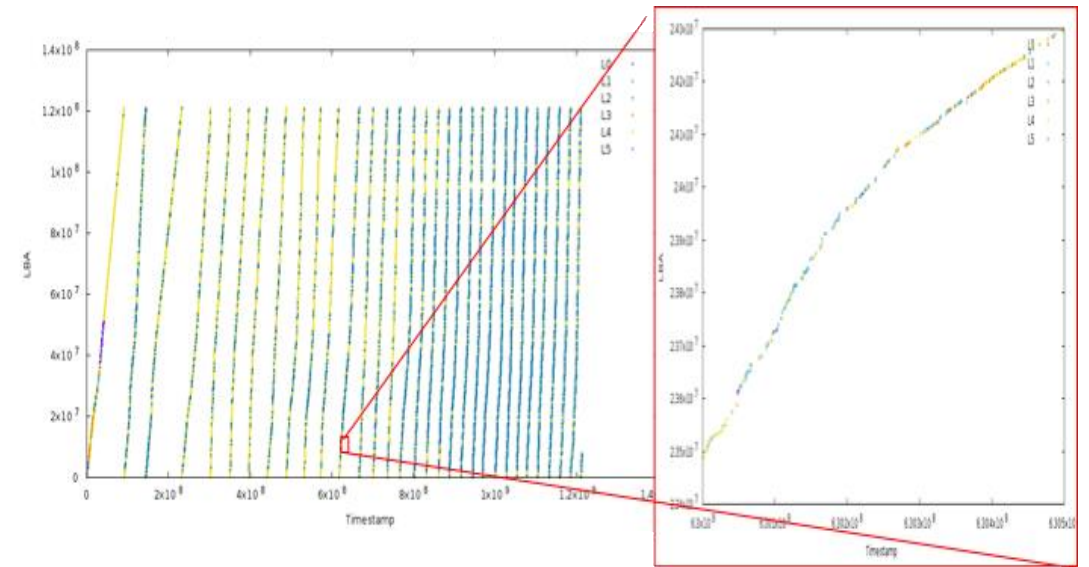
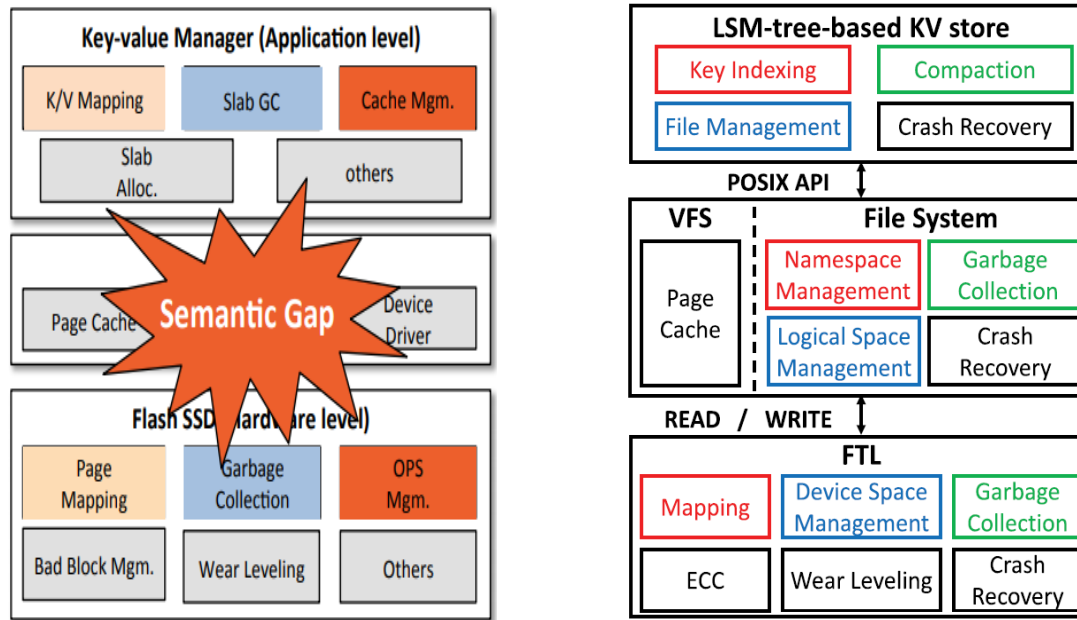
(a) LSM-tree



(b) LevelDB

6. Research Topic

- Semantic Gap (의미론적 간극)
 - Storage SW (I/O Stack) → Layered architecture:
 - 1) FTL at storage
 - 2) FS at kernel
 - 3) KV DB at application level
 - Concern: Redundant Functionalities due to Unawareness



6. Research Topic

- Learned Index

- Instead of data structure, use a model (ML) for indexing
- Less memory footprint, faster lookup, ...

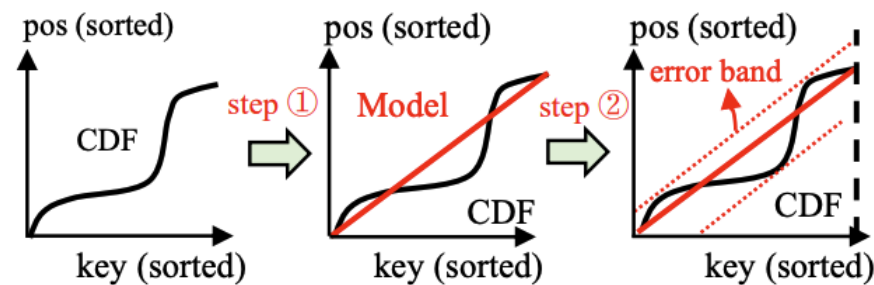


Fig. 4: The workflow of the learned index.

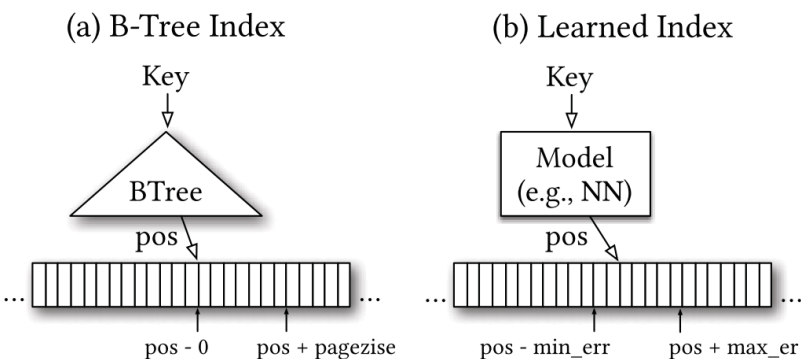


Figure 1: Why B-Trees are models

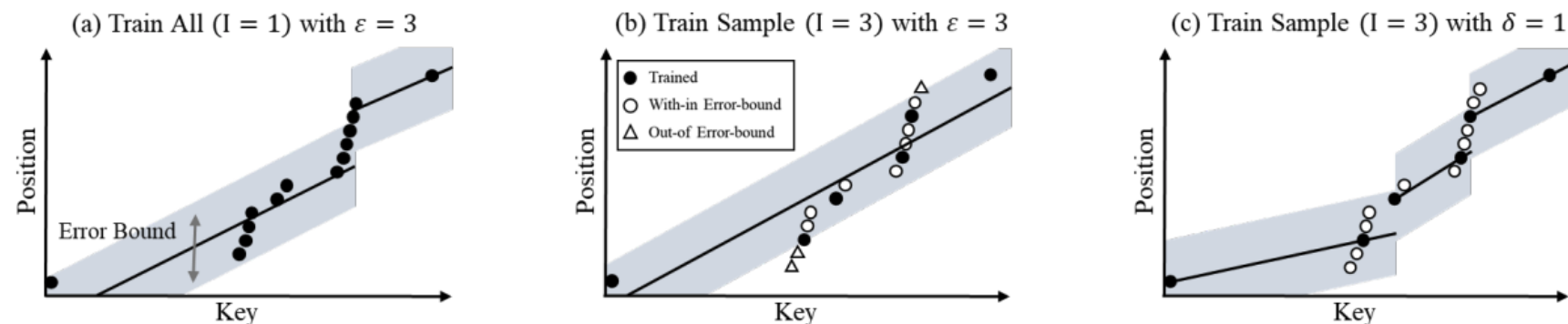


Fig. 2. Impact of sampling on error-bounded piecewise linear approximation.

Ref: Choi, Minguk, Seehwan Yoo, and Jongmoo Choi. "Can Learned Indexes be Built Efficiently? A Deep Dive into Sampling Trade-offs." *Proceedings of the ACM on Management of Data* 2.3 (2024): 1-25.
Ref: Kraska, Tim, et al. "The case for learned index structures." *Proceedings of the 2018 international conference on management of data*. 2018.
Ref: Choi, Minguk, Seehwan Yoo, and Jongmoo Choi. "Can Learned Indexes be Built Efficiently? A Deep Dive into Sampling Trade-offs." *Proceedings of the ACM on Management of Data* 2.3 (2024): 1-25.

6. Research Topic

- LeaFTL, Learned FTL

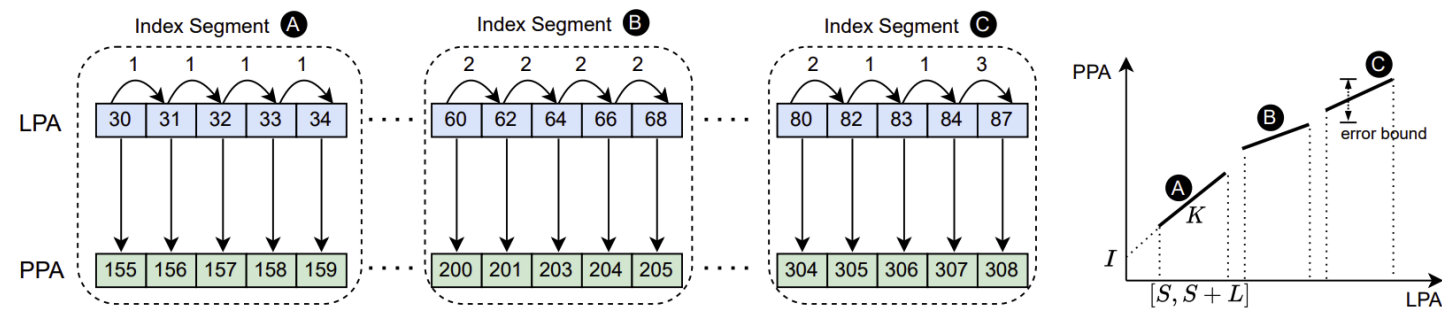


Figure 1: An illustrative example of learning LPA-PPA mappings using piecewise linear regression in LeaFTL. It can learn various patterns of LPA-PPA mappings with guaranteed error bound. Each learned index segment can be represented with (S, L, K, I) , where $[S, S + L]$ denotes the interval of LPAs, K is the slope, and I is the intercept of the index segment.

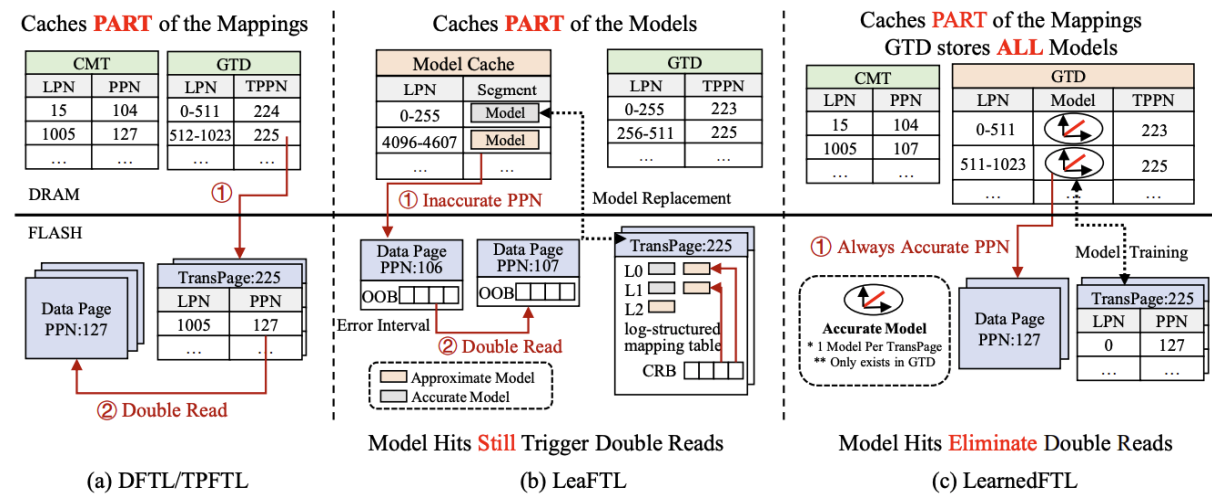


Fig. 1: Representative mapping schemes in flash-based SSDs.

6. Research Topic

- Paper List

- Lee, Sungjin, et al. "{Application-Managed} Flash." *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 2016.
- Park, Jonggyu, and Young Ik Eom. "Fragpicker: A new defragmentation tool for modern storage devices." *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 2021.
- Bjørling, Matias, et al. "{ZNS}: Avoiding the block interface tax for flash-based {SSDs}." *2021 USENIX annual technical conference (USENIX ATC 21)*. 2021.
- Sun, Jinghan, et al. "Leaftl: A learning-based flash translation layer for solid-state drives." *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2023.
- Jun, Yuhun, et al. "We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash {SSDs}." *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. 2024.

7. FEMU

- Paper List

- Lee, Sungjin, et al. "{Application-Managed} Flash." *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 2016.
- Park, Jonggyu, and Young Ik Eom. "Fragpicker: A new defragmentation tool for modern storage devices." *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 2021.
- Bjørling, Matias, et al. "{ZNS}: Avoiding the block interface tax for flash-based {SSDs}." *2021 USENIX annual technical conference (USENIX ATC 21)*. 2021.
- Sun, Jinghan, et al. "Leaftl: A learning-based flash translation layer for solid-state drives." *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2023.
- Jun, Yuhun, et al. "We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash {SSDs}." *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. 2024.

6. Research Topic

- Paper List

- Lee, Sungjin, et al. "{Application-Managed} Flash." *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 2016.
- Park, Jonggyu, and Young Ik Eom. "Fragpicker: A new defragmentation tool for modern storage devices." *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 2021.
- Bjørling, Matias, et al. "{ZNS}: Avoiding the block interface tax for flash-based {SSDs}." *2021 USENIX annual technical conference (USENIX ATC 21)*. 2021.
- Sun, Jinghan, et al. "Leaftrl: A learning-based flash translation layer for solid-state drives." *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2023.
- Jiao, Ziyang, et al. "The Design and Implementation of a {Capacity-Variant} Storage System." *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. 2024.
- Jun, Yuhun, et al. "We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash {SSDs}." *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. 2024.

6. FEMU

- Link : <https://github.com/MoatLab/FEMU>
- Paper : <https://www.usenix.org/conference/fast18/presentation/li>

2026 여름 방학 FTL Study

Basic Lecture

Thank you!
Q & A ?

2026.07.21

Presentation by Choi, Gunhee

choi_gunhee@dankook.ac.kr